

Experiment 7

Bagging, Boosting and Stacked Ensemble Models

Name :Jayasri M
Regno:3122247001025

1 Aim

To implement and compare Bagging, Boosting, and Stacking ensemble learning techniques using the Wisconsin Breast Cancer Dataset and analyze their classification performance.

2 Objectives

1. To understand the concept of ensemble learning.
2. To implement Bagging using Decision Trees.
3. To implement Boosting techniques such as AdaBoost and Gradient Boosting.
4. To build a Stacking model using multiple base classifiers.
5. To evaluate and compare the performance of different ensemble models.

3 Dataset Description

The Wisconsin Diagnostic Breast Cancer Dataset was used for this experiment. The dataset contains 569 samples with 30 numerical features extracted from breast cancer cell images. The target variable contains two classes:

- Malignant (M)
- Benign (B)

4 Preprocessing Steps

1. Loaded the dataset into a Pandas DataFrame.
2. Examined the dataset using `head()`, `info()`, and `describe()`.
3. Checked for missing values.
4. Converted the target class into numerical format.

5. Performed Exploratory Data Analysis (EDA).
6. Separated the dataset into features (X) and target variable (y).
7. Split the dataset into training and testing sets using an 80:20 ratio.
8. Applied feature scaling where required.

```
In [64]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, GridSearchCV, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import BaggingClassifier, AdaBoostClassifier, GradientBoostingC
from sklearn.svm import SVC
from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, co
from sklearn.metrics import roc_curve
from sklearn.metrics import roc_auc_score
```

```
In [39]: df=pd.read_csv('C:/Users/DHARSHINI/Downloads/sem5/machine-learning/experiment-6/dat
df.head()
```

```
Out[39]:
```

	0	1	2	3	4	5	6	7	8	9	...	22
0	842302	M	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.14710	...	25.38
1	842517	M	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.07017	...	24.99
2	84300903	M	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.12790	...	23.57
3	84348301	M	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.10520	...	14.91
4	84358402	M	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.10430	...	22.54

5 rows × 32 columns



```
In [40]: print(df.shape)
print(df.columns)
print(df.info())
print(df.describe())
print("\n\nNULL values :", df.isnull().sum().sum())
```

```
(569, 32)
Index([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16, 17,
        18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31],
      dtype='int64')
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 32 columns):
#   Column  Non-Null Count  Dtype
---  -
0    0      569 non-null    int64
1    1      569 non-null    object
2    2      569 non-null    float64
3    3      569 non-null    float64
4    4      569 non-null    float64
5    5      569 non-null    float64
6    6      569 non-null    float64
7    7      569 non-null    float64
8    8      569 non-null    float64
9    9      569 non-null    float64
10   10      569 non-null    float64
11   11      569 non-null    float64
12   12      569 non-null    float64
13   13      569 non-null    float64
14   14      569 non-null    float64
15   15      569 non-null    float64
16   16      569 non-null    float64
17   17      569 non-null    float64
18   18      569 non-null    float64
19   19      569 non-null    float64
20   20      569 non-null    float64
21   21      569 non-null    float64
22   22      569 non-null    float64
23   23      569 non-null    float64
24   24      569 non-null    float64
25   25      569 non-null    float64
26   26      569 non-null    float64
27   27      569 non-null    float64
28   28      569 non-null    float64
29   29      569 non-null    float64
30   30      569 non-null    float64
31   31      569 non-null    float64
dtypes: float64(30), int64(1), object(1)
memory usage: 142.4+ KB
None
```

	0	2	3	4	5	\
count	5.690000e+02	569.000000	569.000000	569.000000	569.000000	
mean	3.037183e+07	14.127292	19.289649	91.969033	654.889104	
std	1.250206e+08	3.524049	4.301036	24.298981	351.914129	
min	8.670000e+03	6.981000	9.710000	43.790000	143.500000	
25%	8.692180e+05	11.700000	16.170000	75.170000	420.300000	
50%	9.060240e+05	13.370000	18.840000	86.240000	551.100000	
75%	8.813129e+06	15.780000	21.800000	104.100000	782.700000	
max	9.113205e+08	28.110000	39.280000	188.500000	2501.000000	

	6	7	8	9	10	...	\
count	569.000000	569.000000	569.000000	569.000000	569.000000	...	

mean	0.096360	0.104341	0.088799	0.048919	0.181162	...
std	0.014064	0.052813	0.079720	0.038803	0.027414	...
min	0.052630	0.019380	0.000000	0.000000	0.106000	...
25%	0.086370	0.064920	0.029560	0.020310	0.161900	...
50%	0.095870	0.092630	0.061540	0.033500	0.179200	...
75%	0.105300	0.130400	0.130700	0.074000	0.195700	...
max	0.163400	0.345400	0.426800	0.201200	0.304000	...

	22	23	24	25	26 \
count	569.000000	569.000000	569.000000	569.000000	569.000000
mean	16.269190	25.677223	107.261213	880.583128	0.132369
std	4.833242	6.146258	33.602542	569.356993	0.022832
min	7.930000	12.020000	50.410000	185.200000	0.071170
25%	13.010000	21.080000	84.110000	515.300000	0.116600
50%	14.970000	25.410000	97.660000	686.500000	0.131300
75%	18.790000	29.720000	125.400000	1084.000000	0.146000
max	36.040000	49.540000	251.200000	4254.000000	0.222600

	27	28	29	30	31
count	569.000000	569.000000	569.000000	569.000000	569.000000
mean	0.254265	0.272188	0.114606	0.290076	0.083946
std	0.157336	0.208624	0.065732	0.061867	0.018061
min	0.027290	0.000000	0.000000	0.156500	0.055040
25%	0.147200	0.114500	0.064930	0.250400	0.071460
50%	0.211900	0.226700	0.099930	0.282200	0.080040
75%	0.339100	0.382900	0.161400	0.317900	0.092080
max	1.058000	1.252000	0.291000	0.663800	0.207500

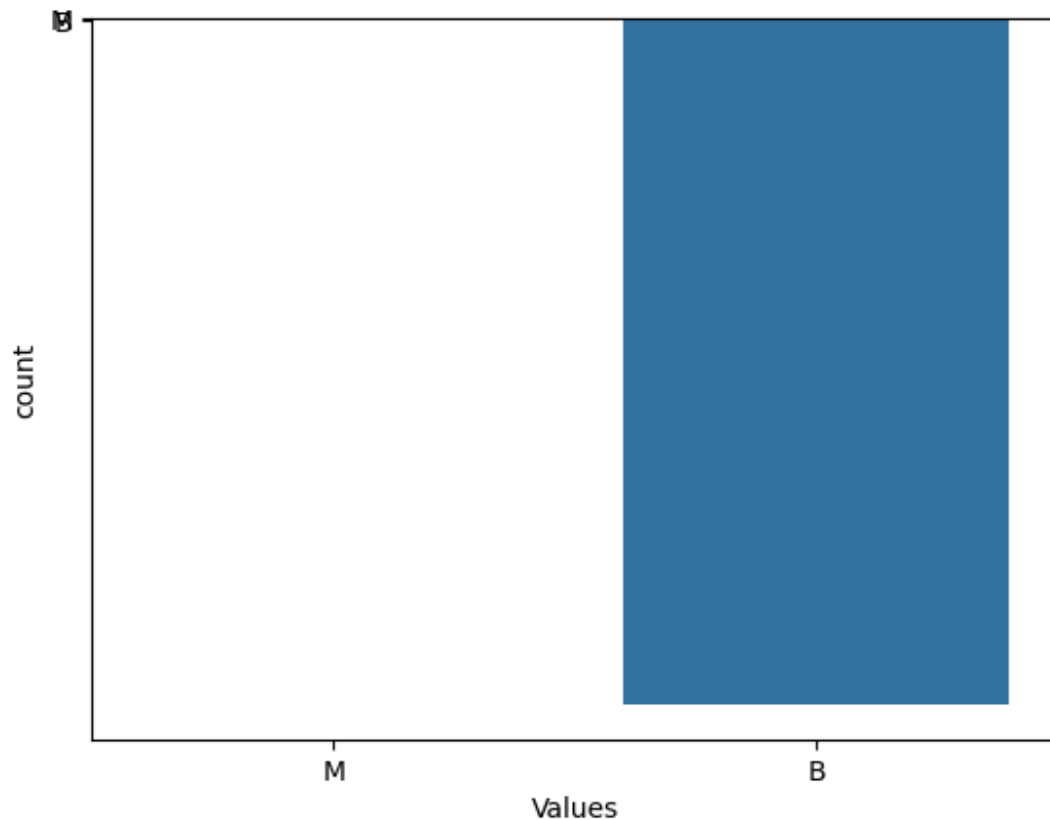
[8 rows x 31 columns]

NULL values : 0

Class Distribution

```
In [41]: def count_plot(col):
          plt.xlabel('Values')
          sns.countplot(x=col,data=df)
          plt.show()
```

```
In [42]: count_plot(df[1])
          df[1].value_counts()
```



Out[42]: 1
 B 357
 M 212
 Name: count, dtype: int64

Box plot

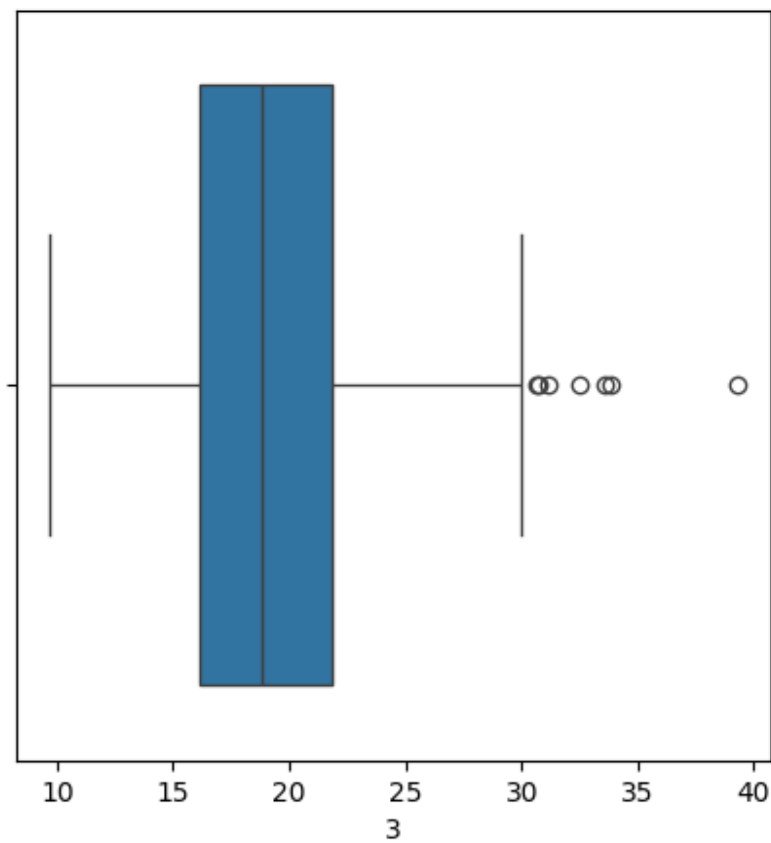
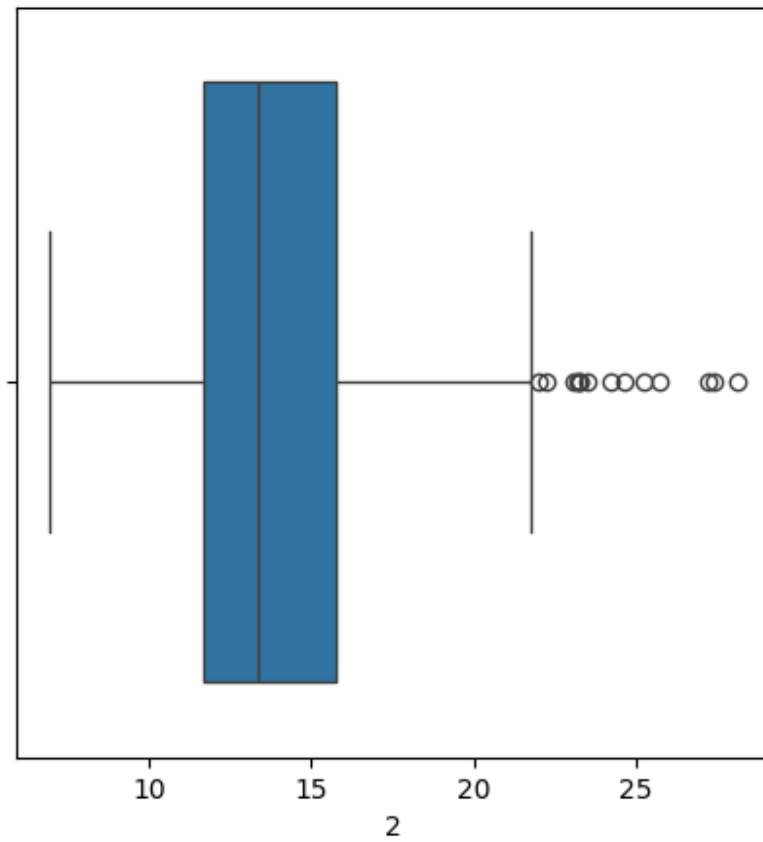
```
In [43]: def box_plot(col):
plt.figure(figsize=(5,5))
sns.boxplot(x=df[col])
plt.xlabel(col)
plt.show()
```

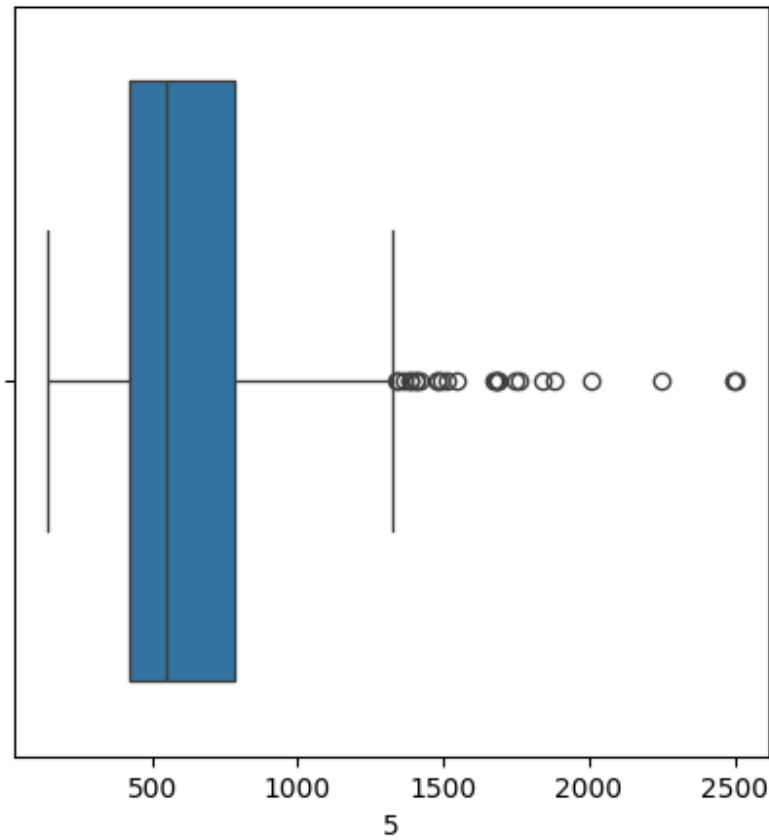
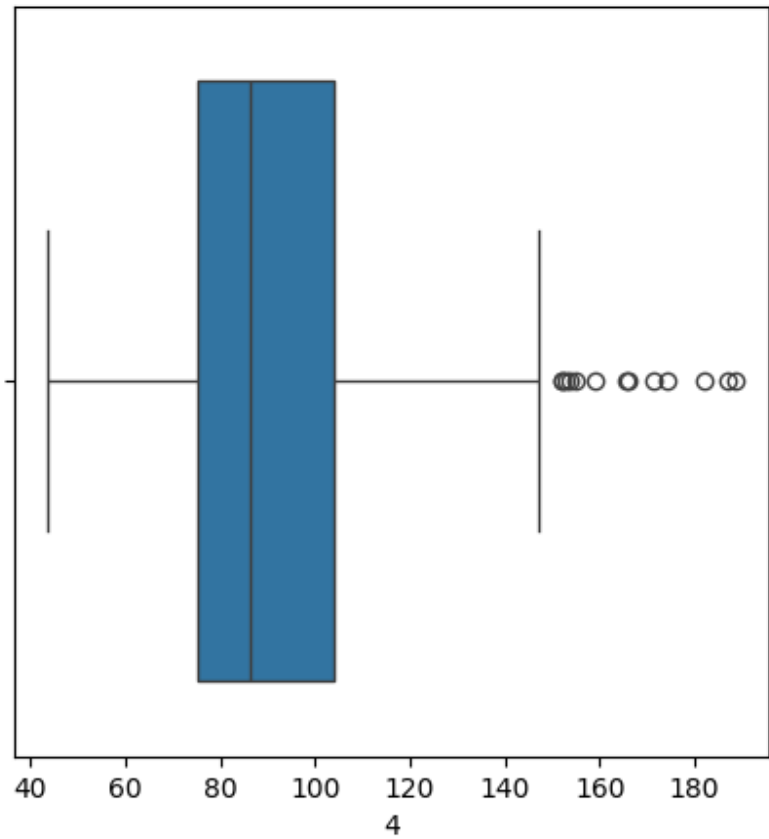
```
In [44]: def histogram(col):
plt.figure(figsize=(5,5))
df[col].hist()
plt.xlabel(col)
plt.show()
```

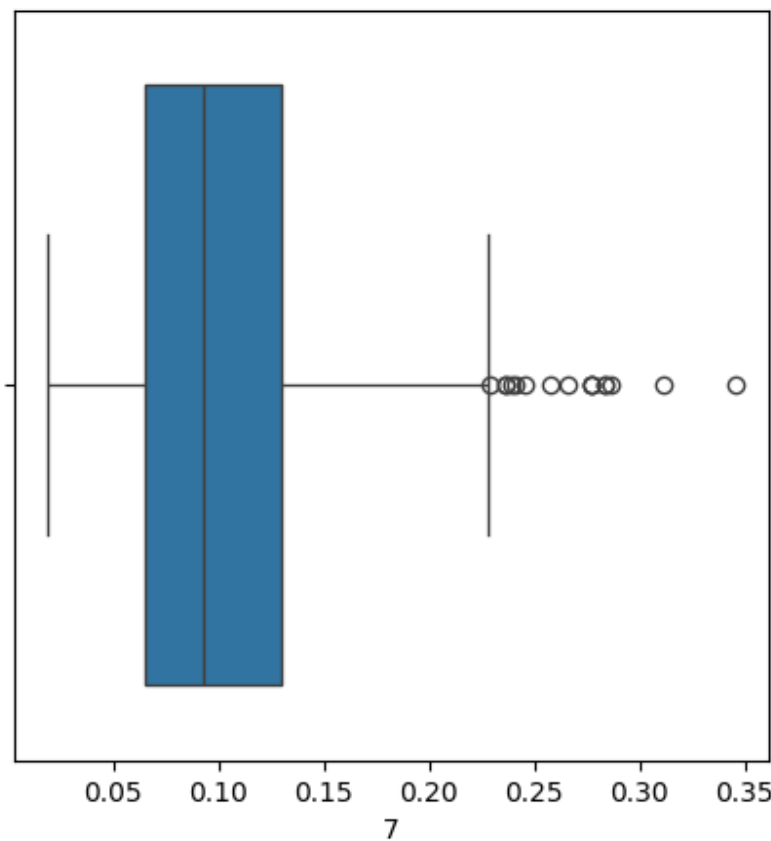
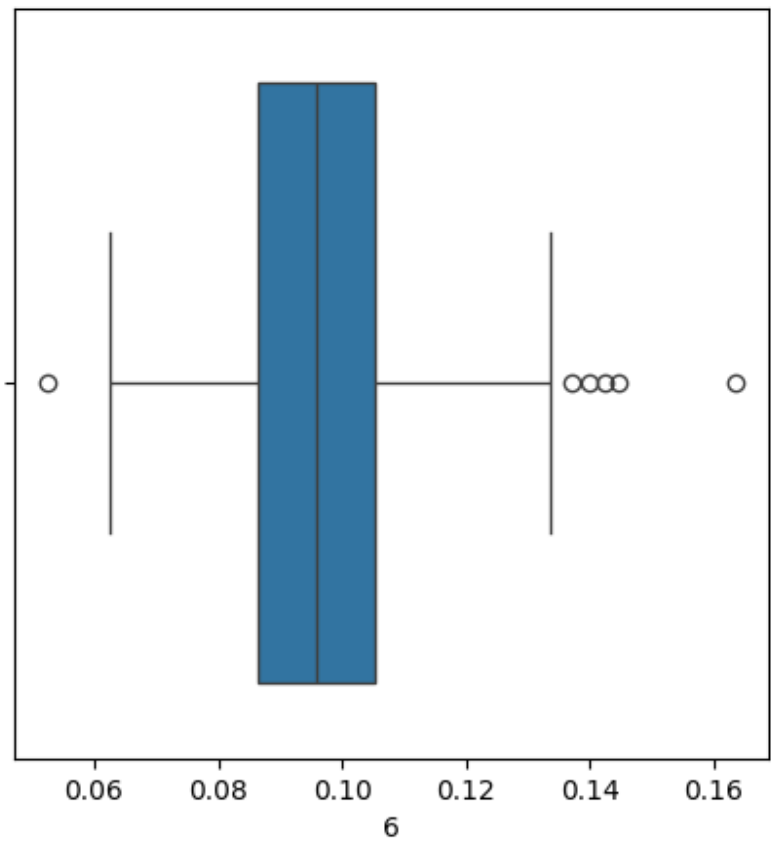
```
In [45]: def correlation_analysis(columns):
plt.figure(figsize=(15,15))
sns.heatmap(columns.corr(),cmap='coolwarm')
plt.show()
```

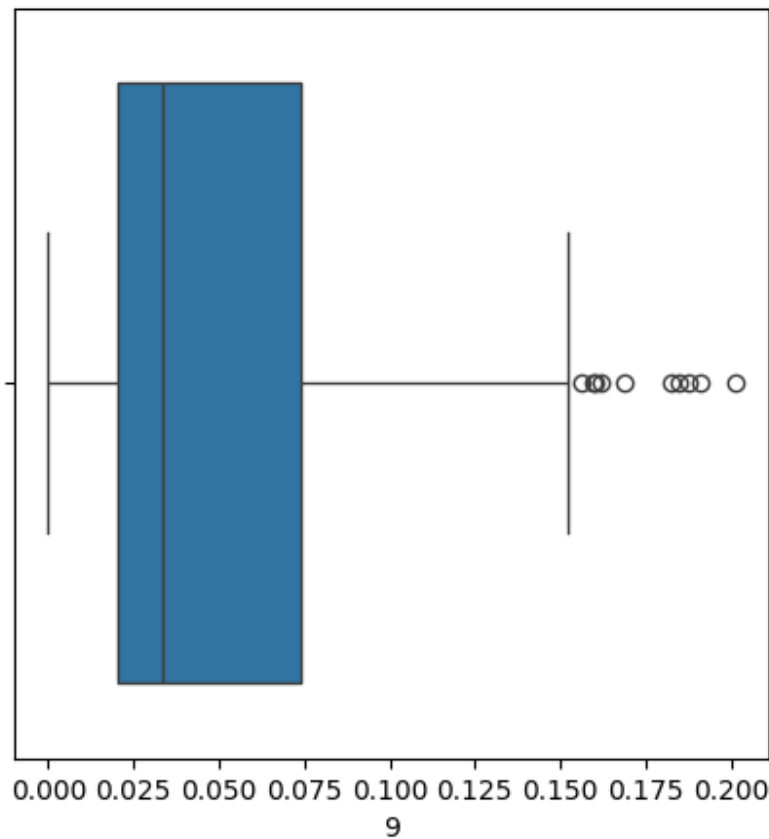
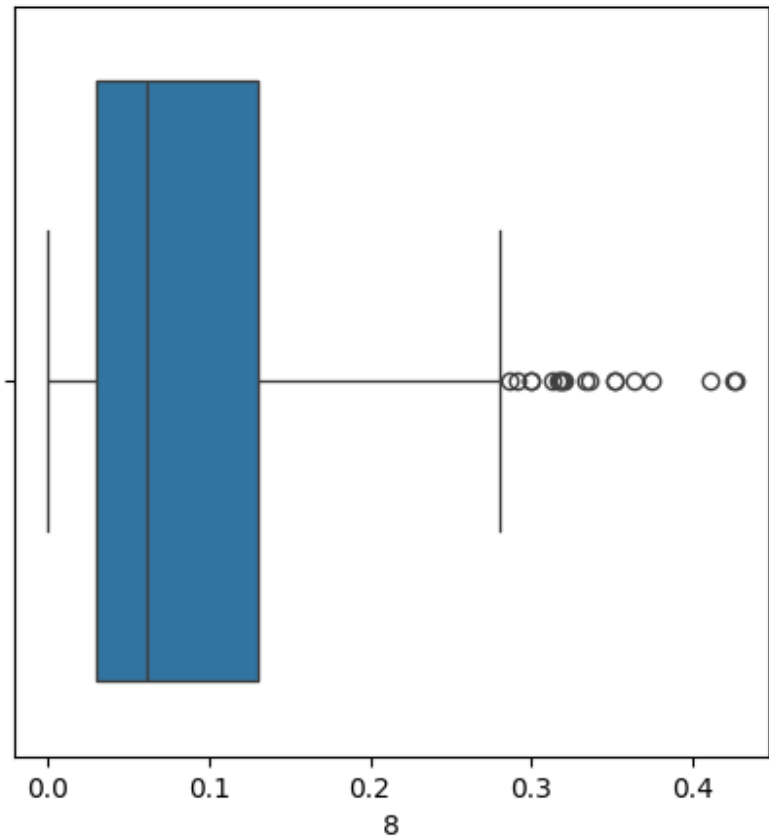
```
In [46]: columns=df.drop(columns=[0,1])
```

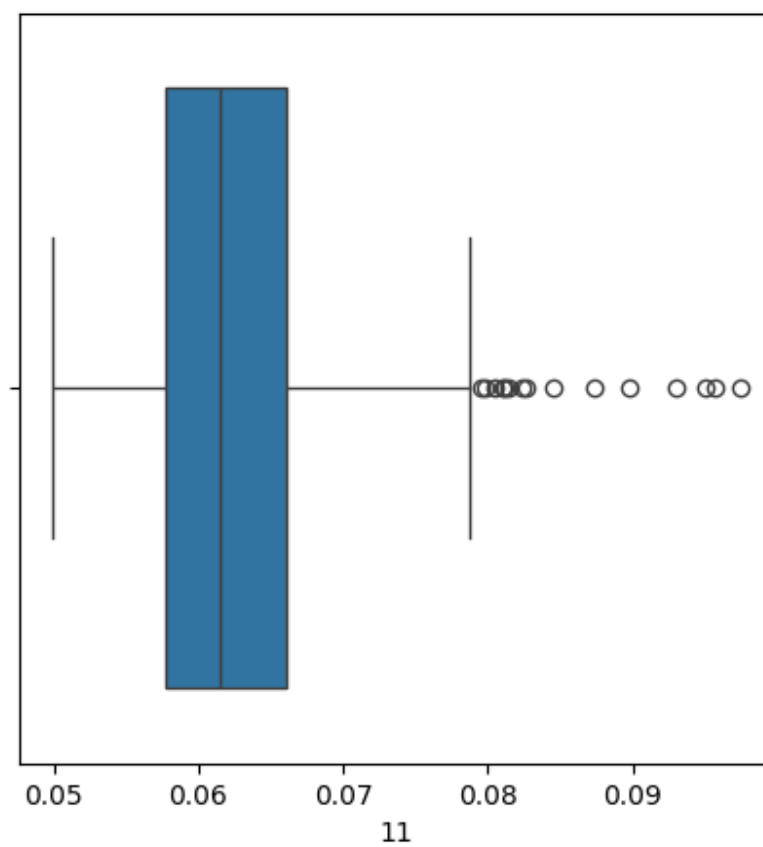
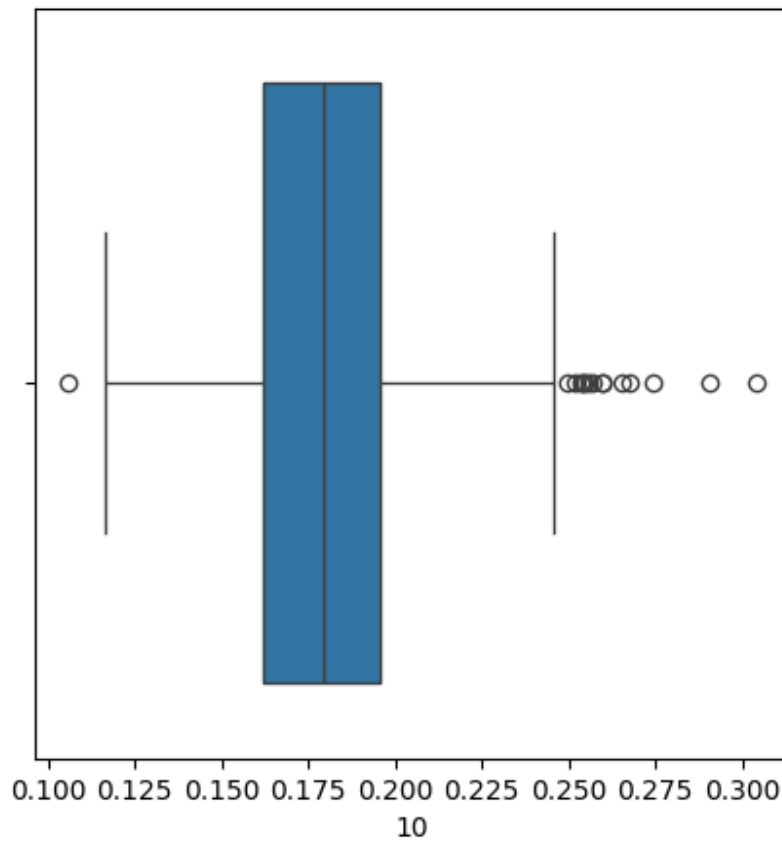
```
for col in columns:  
    box_plot(col)
```

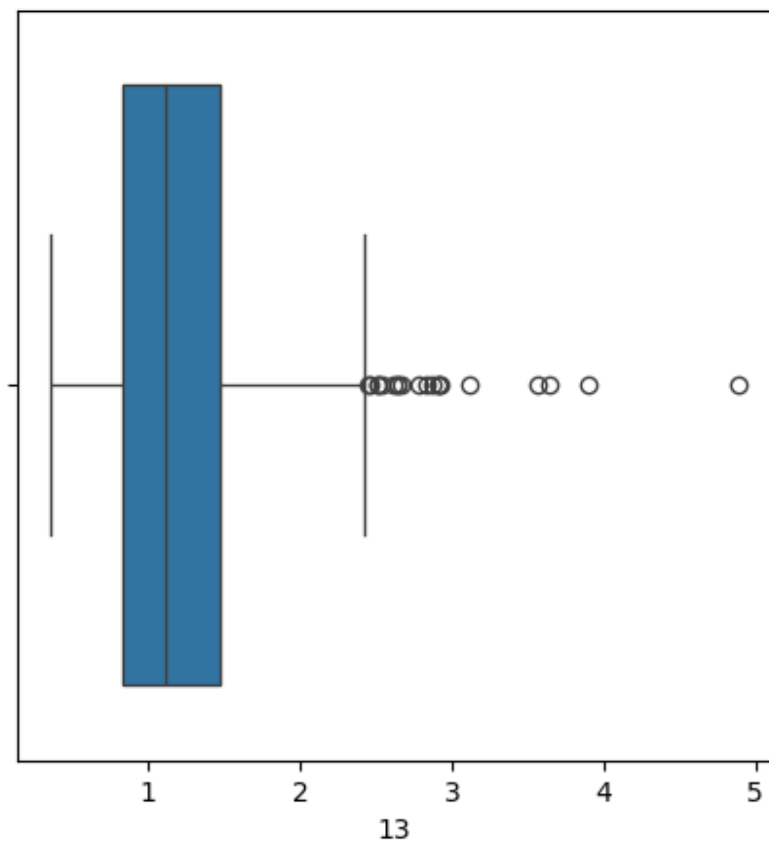
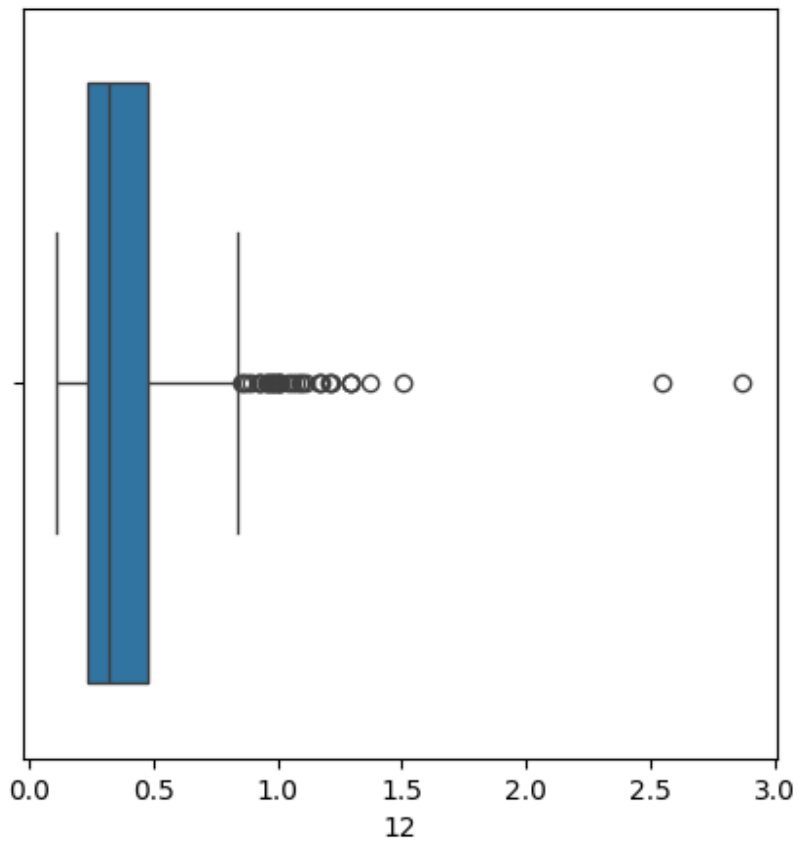


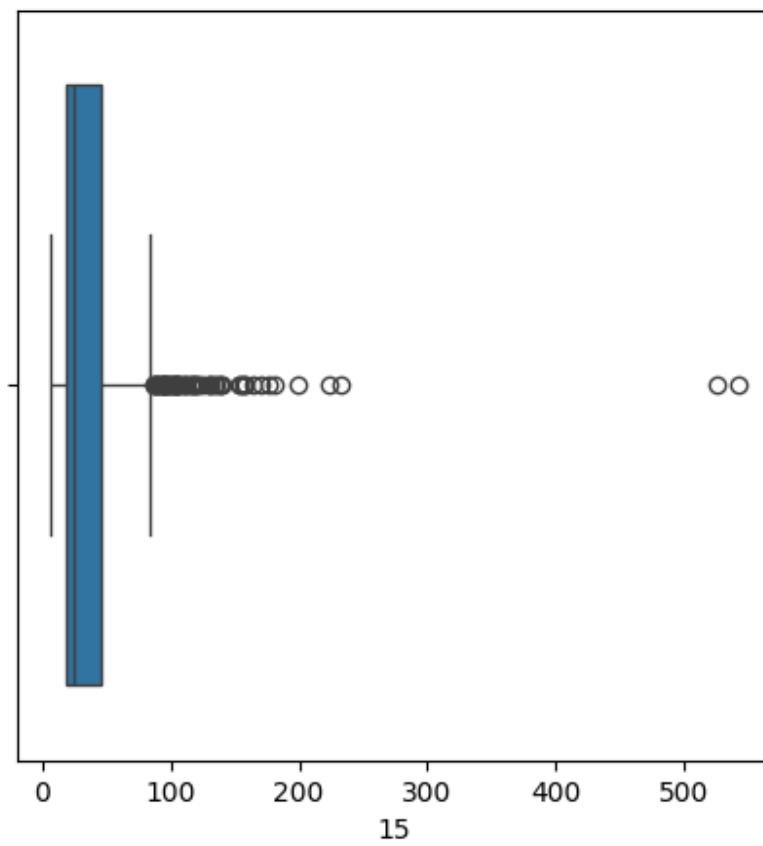
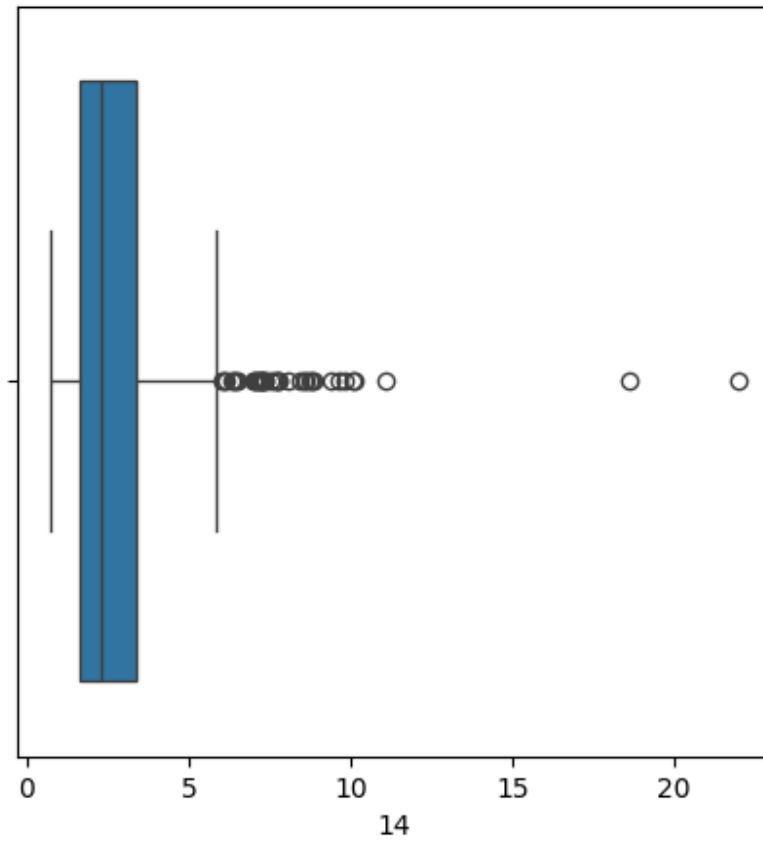


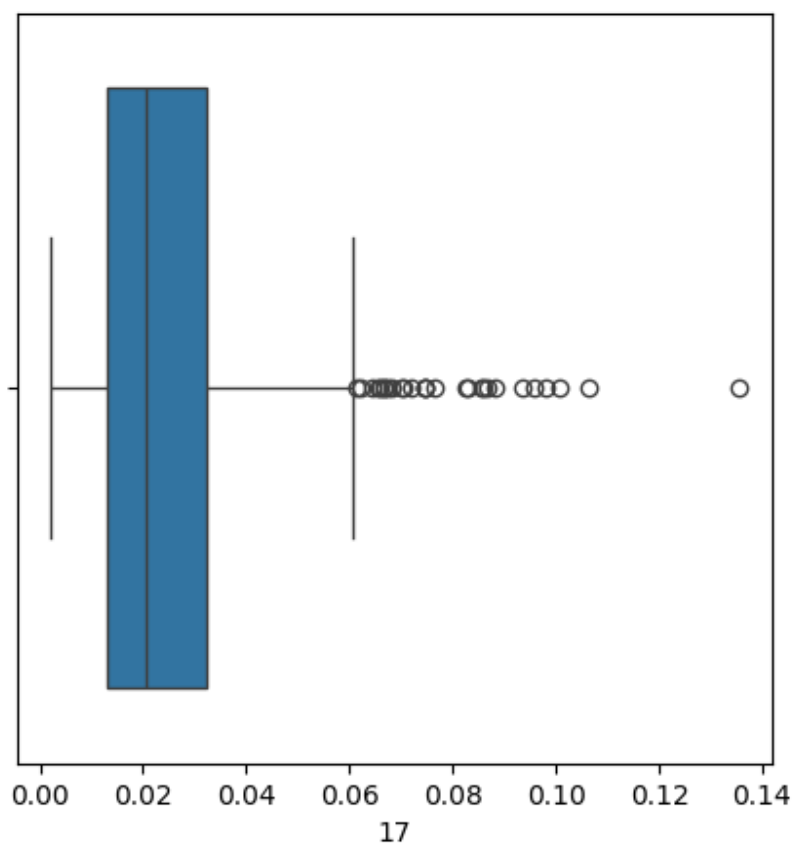
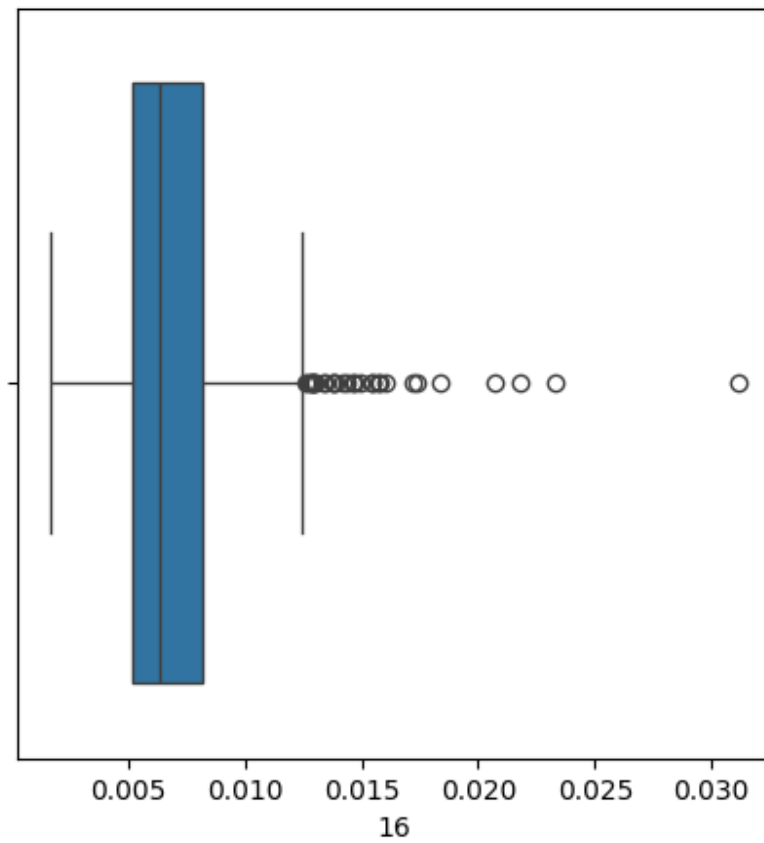


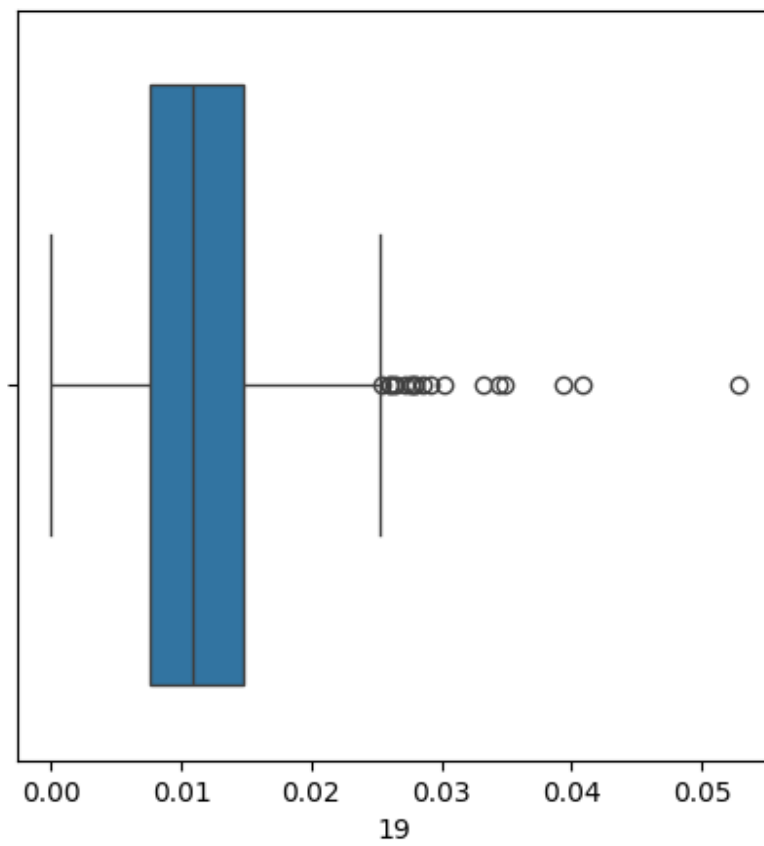
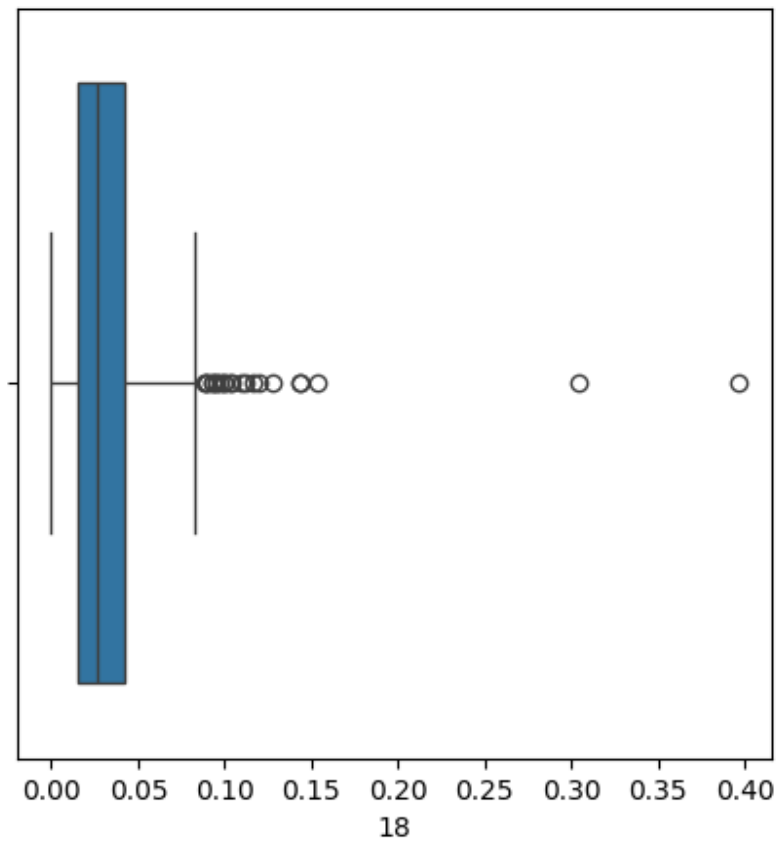


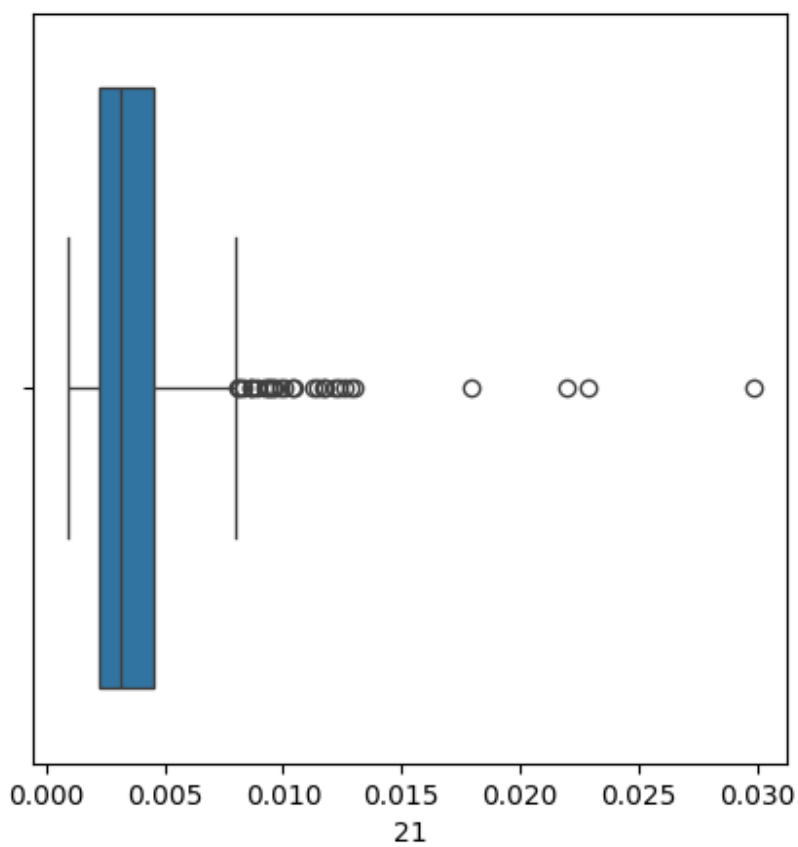
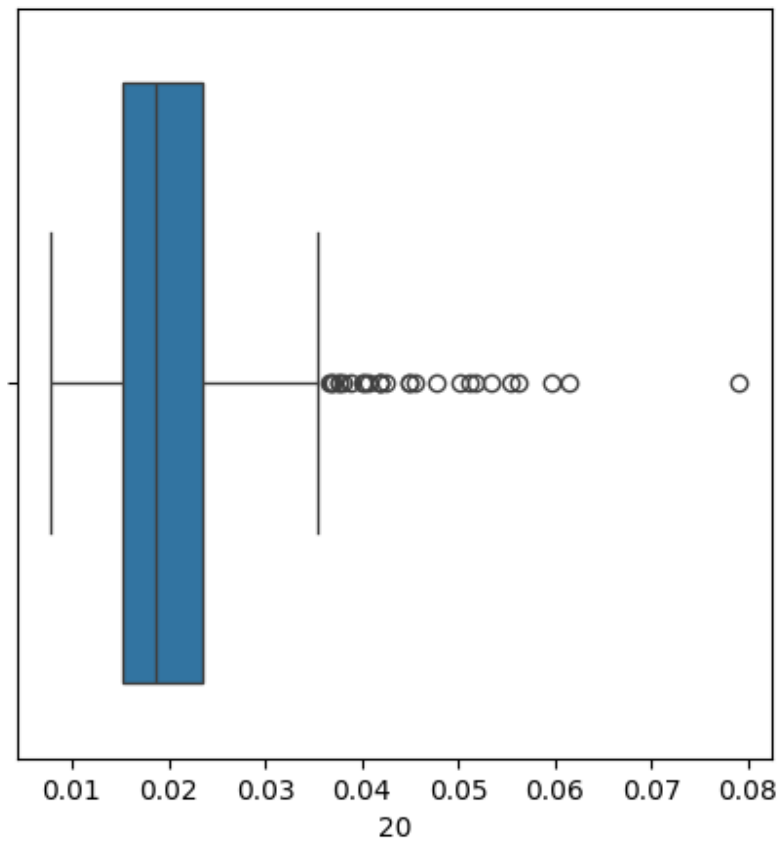


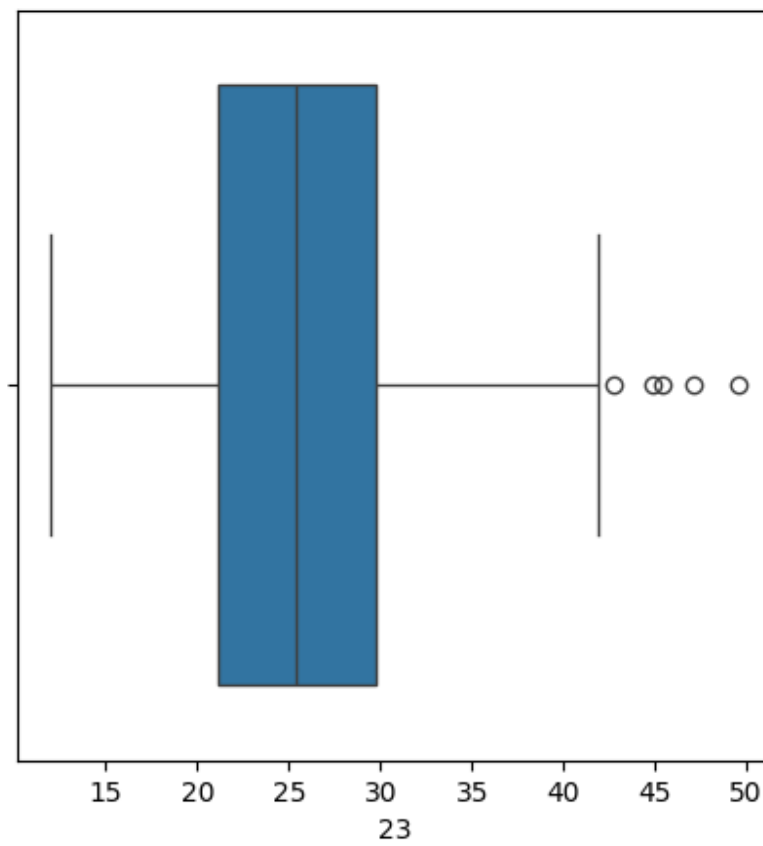
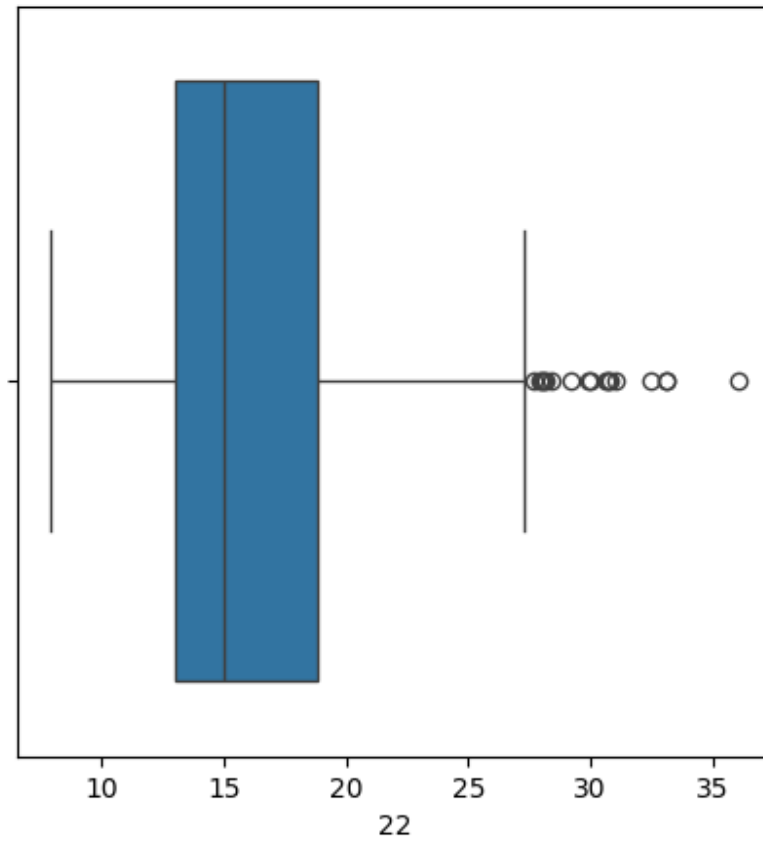


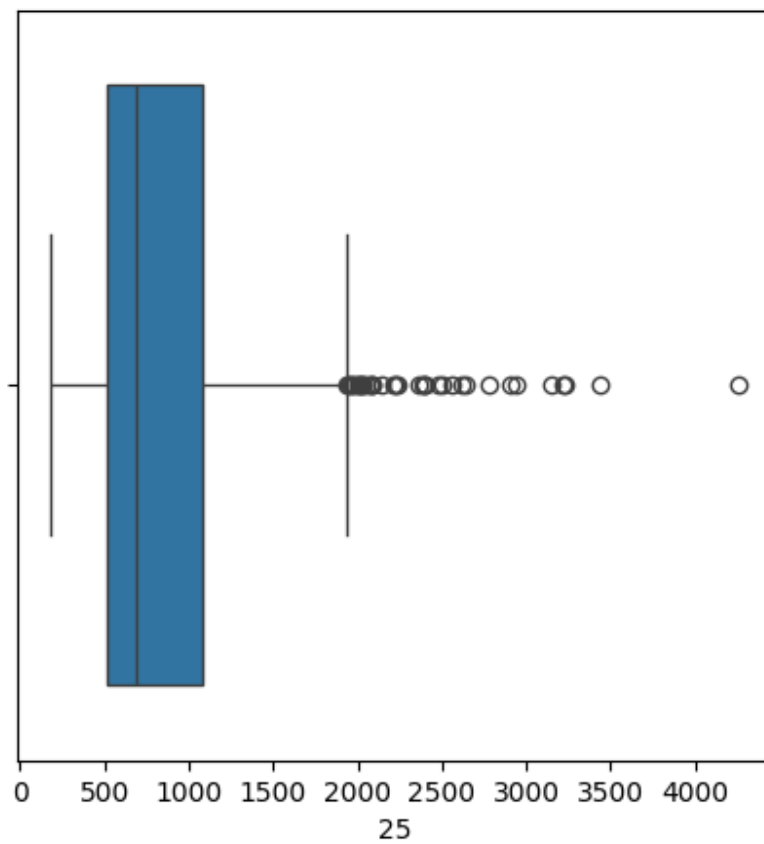
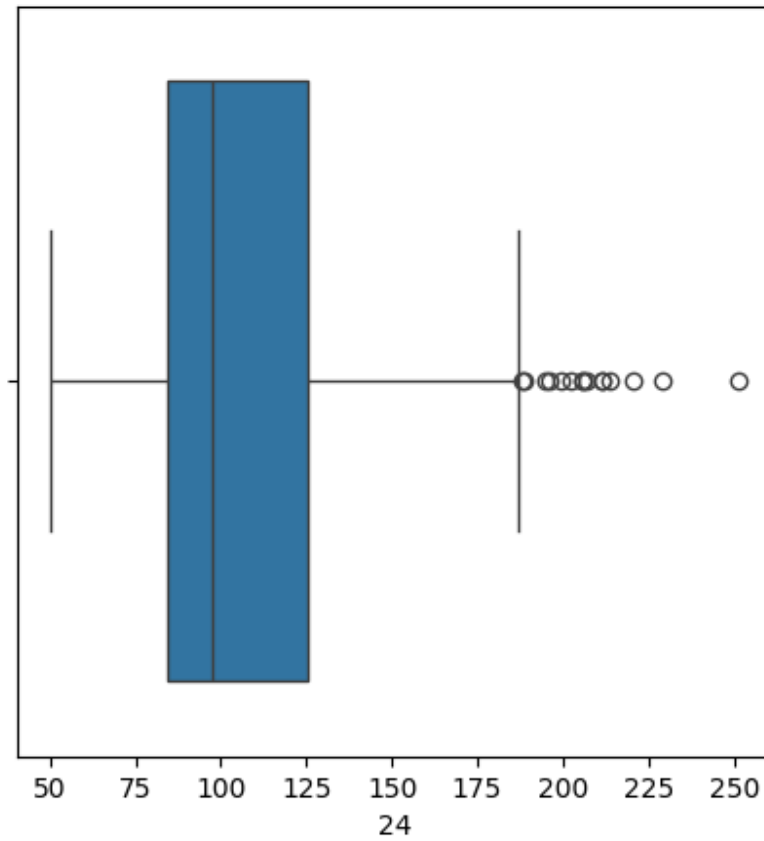


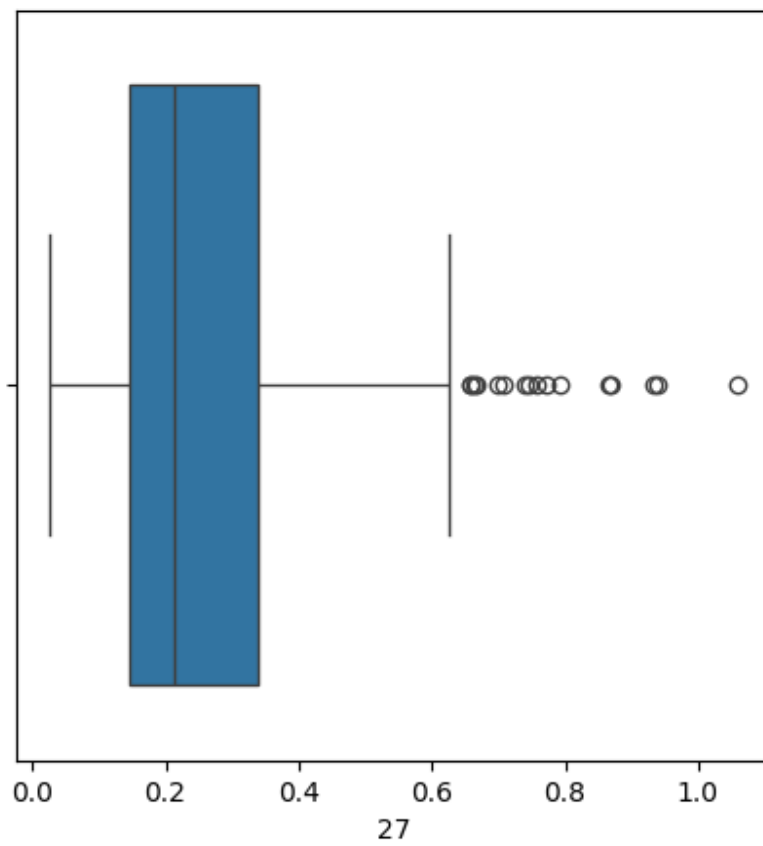
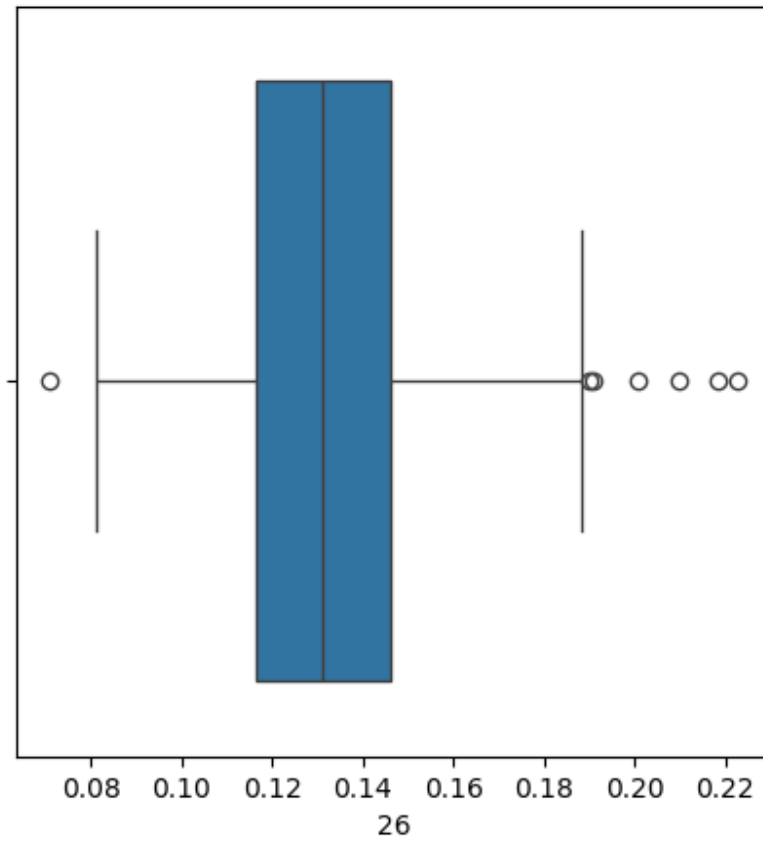


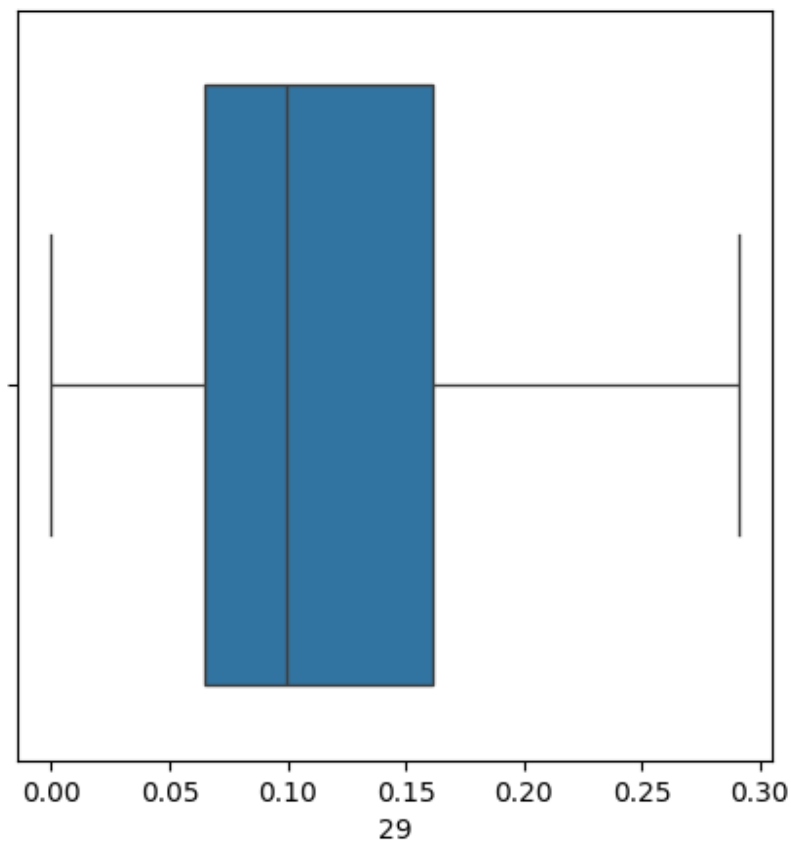
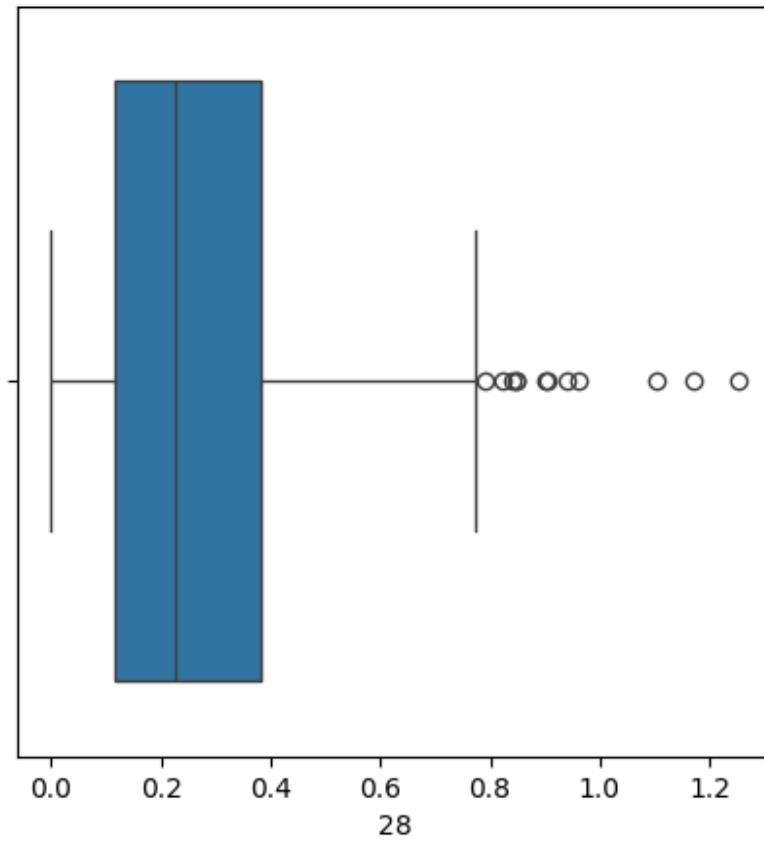


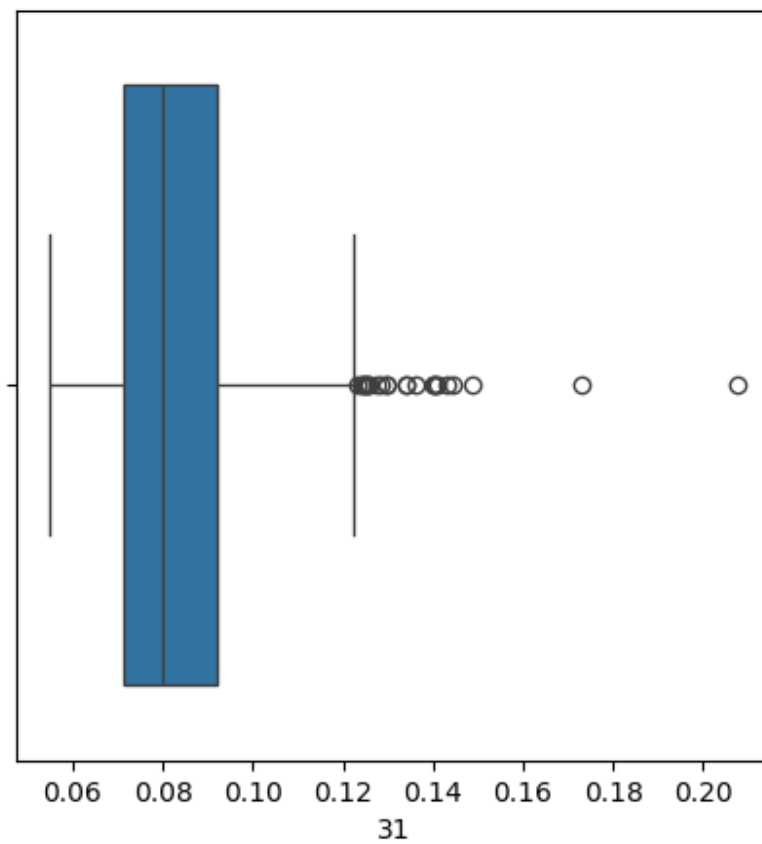
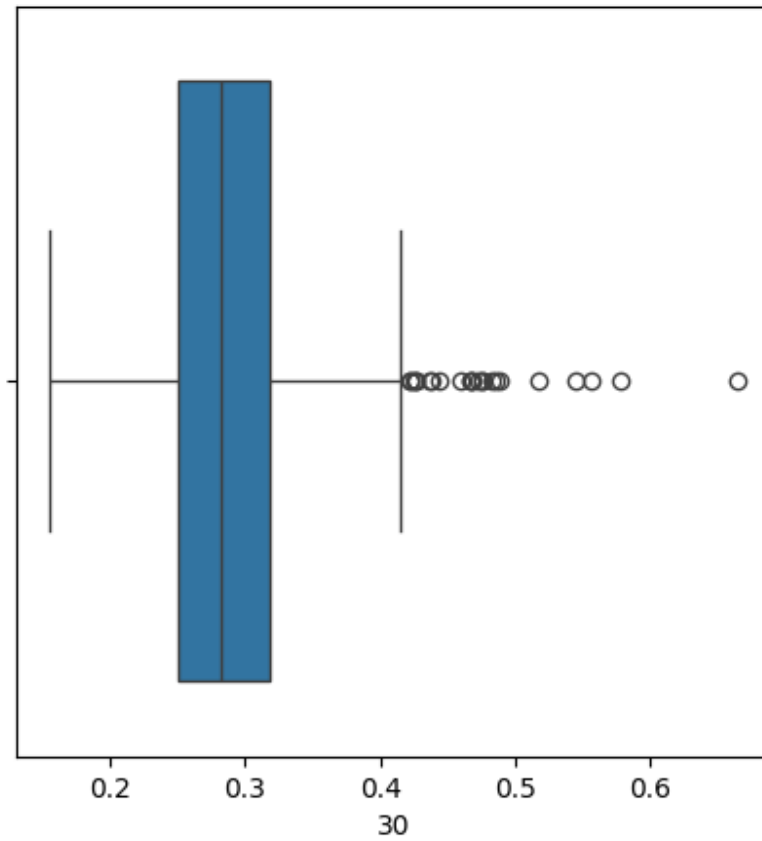




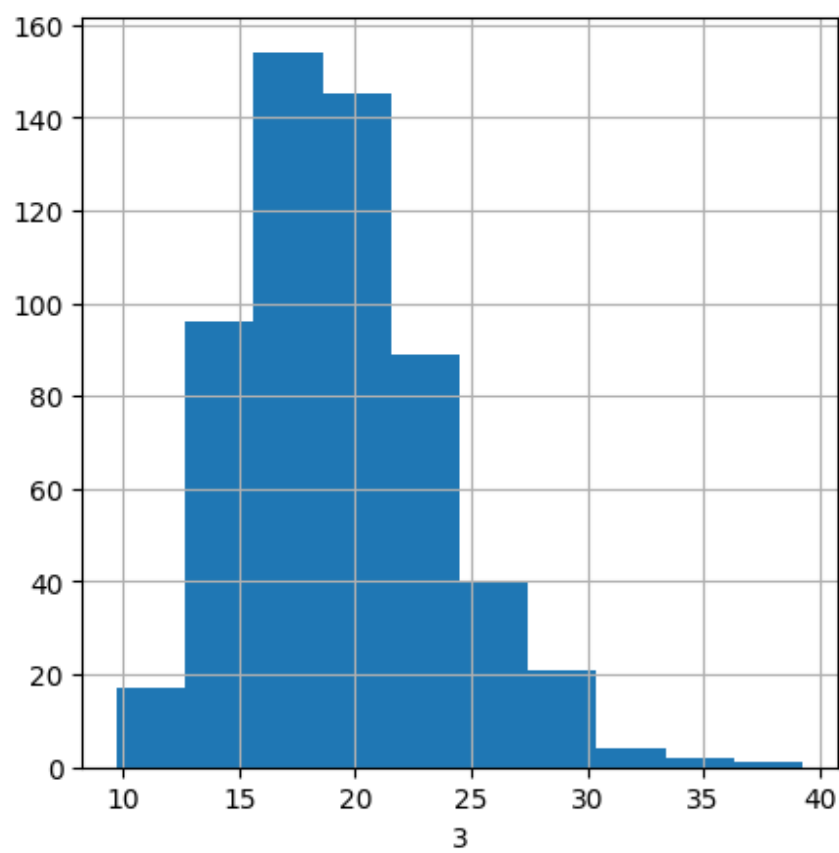
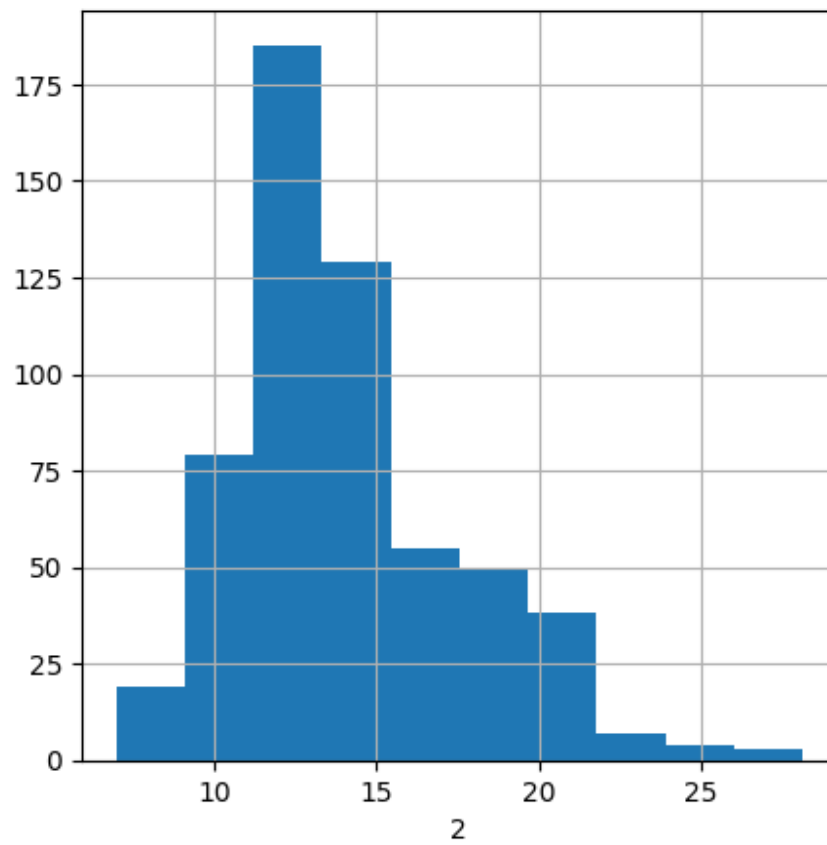


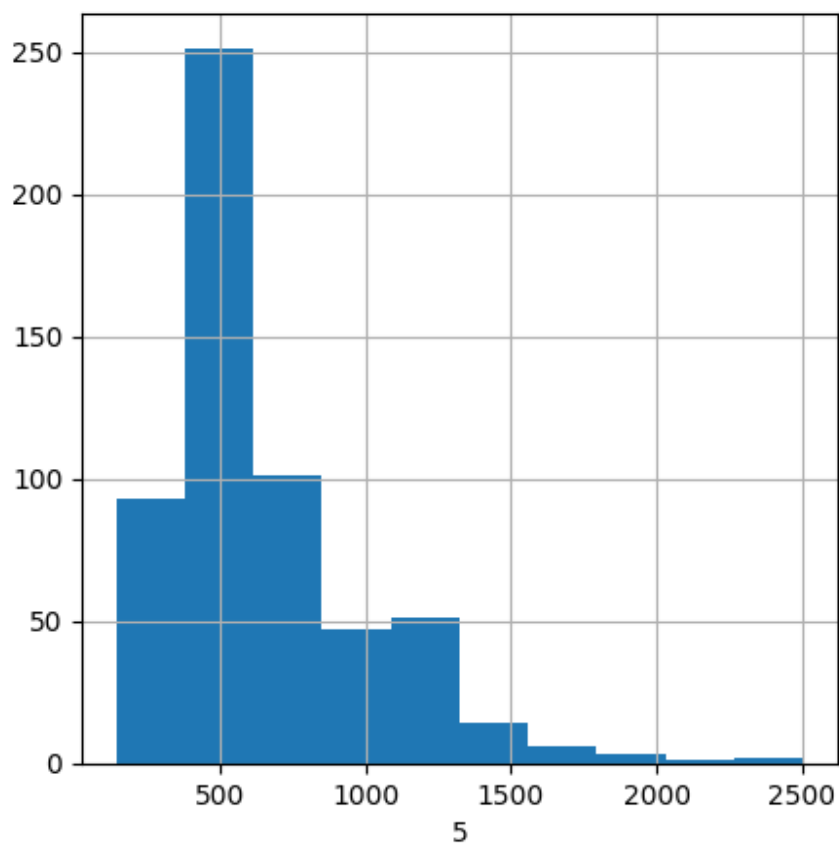
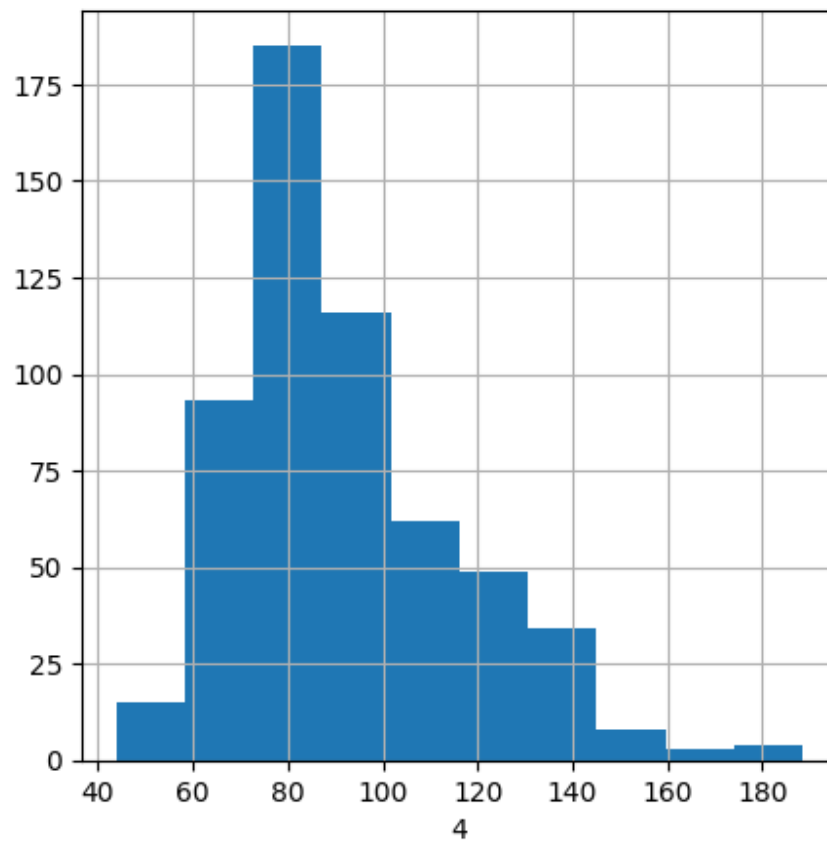


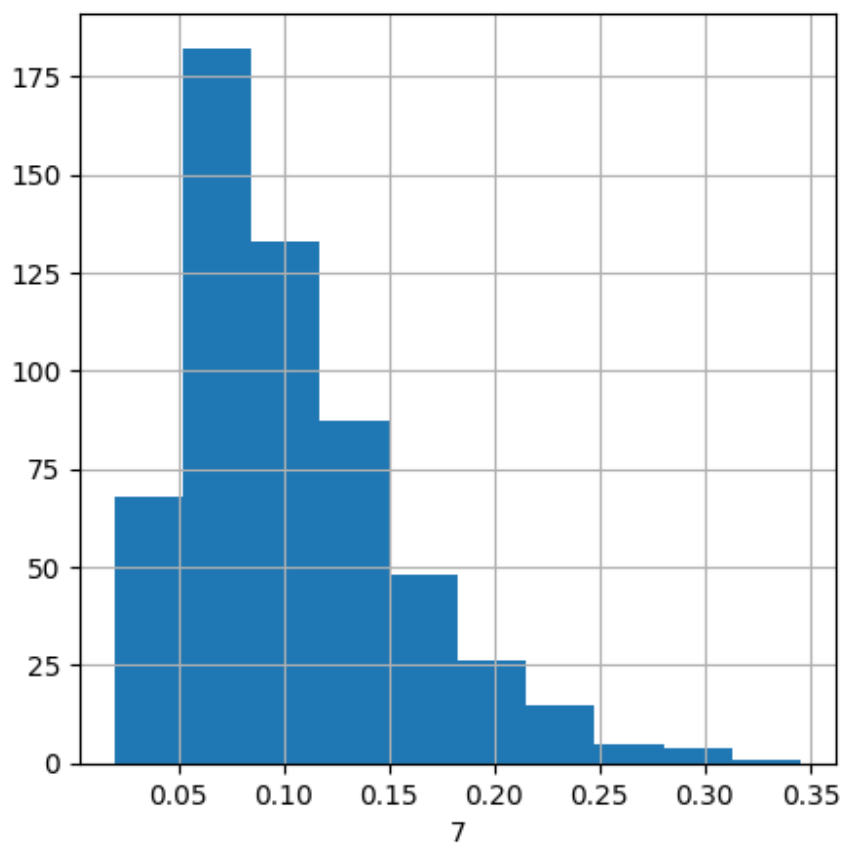
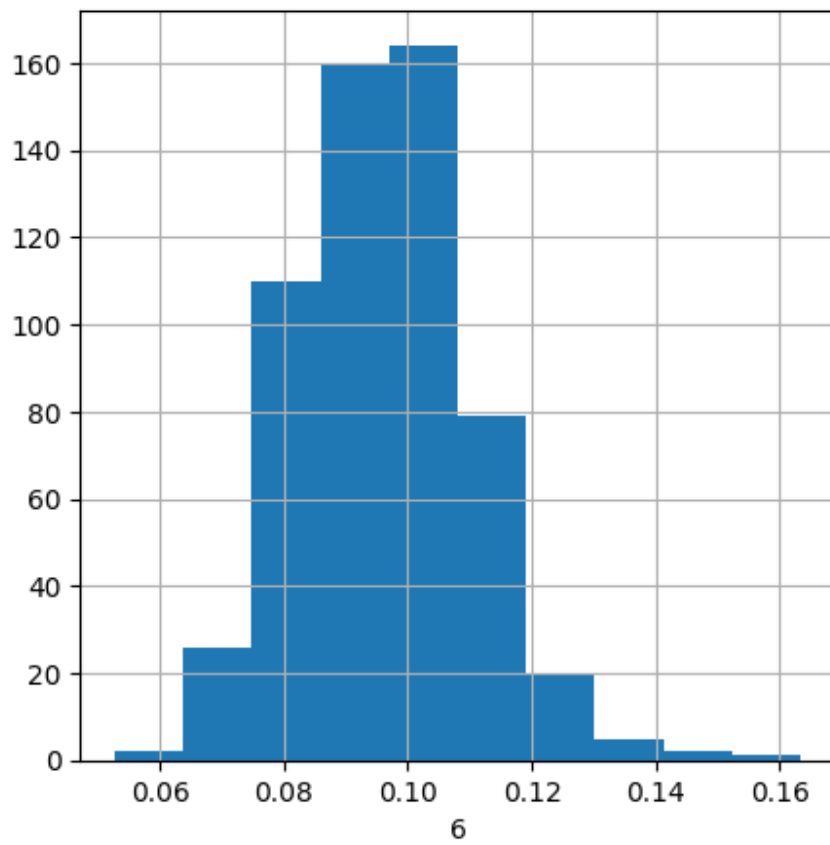


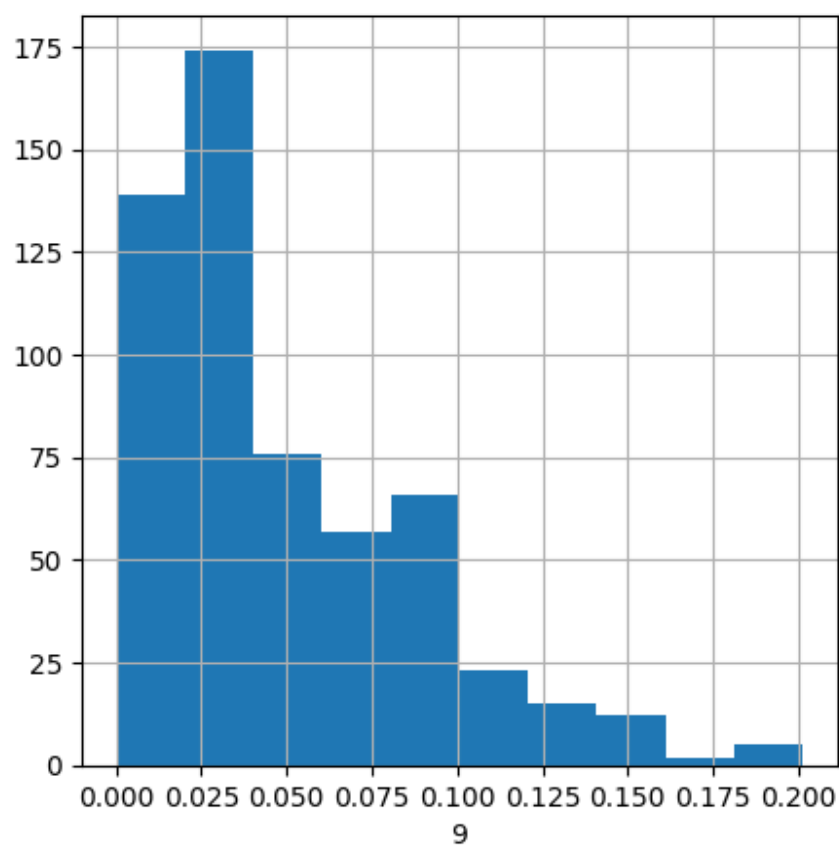
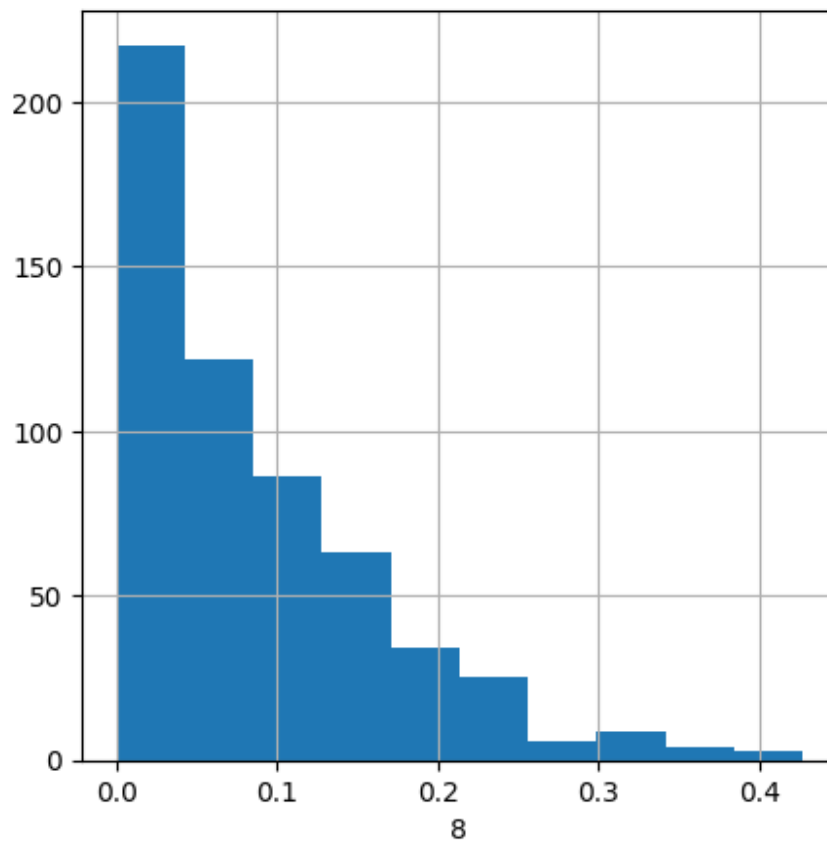


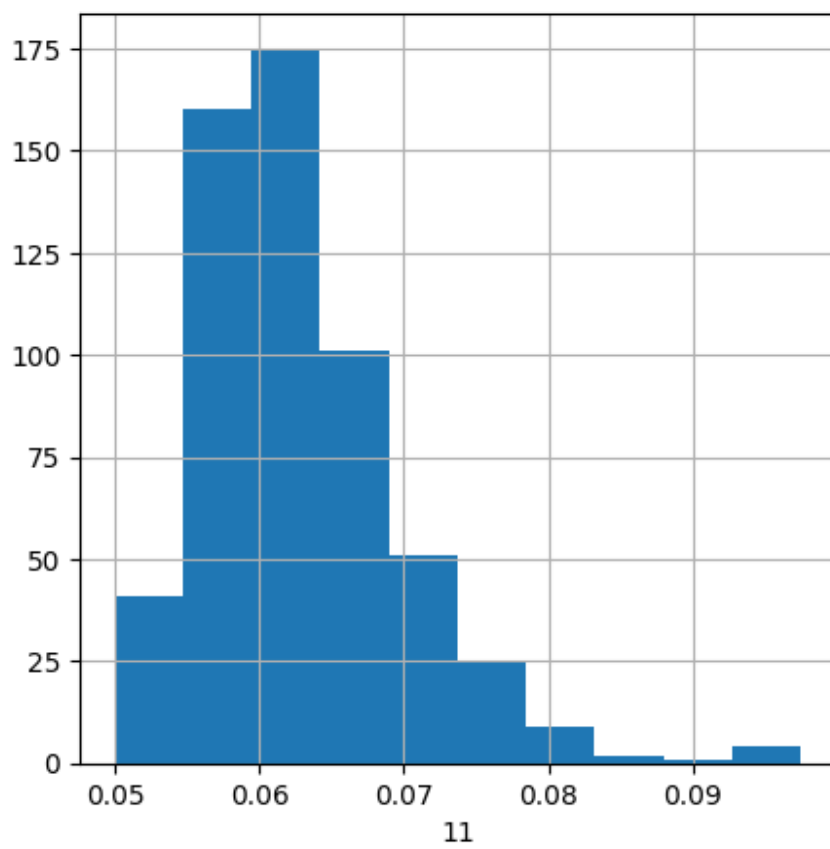
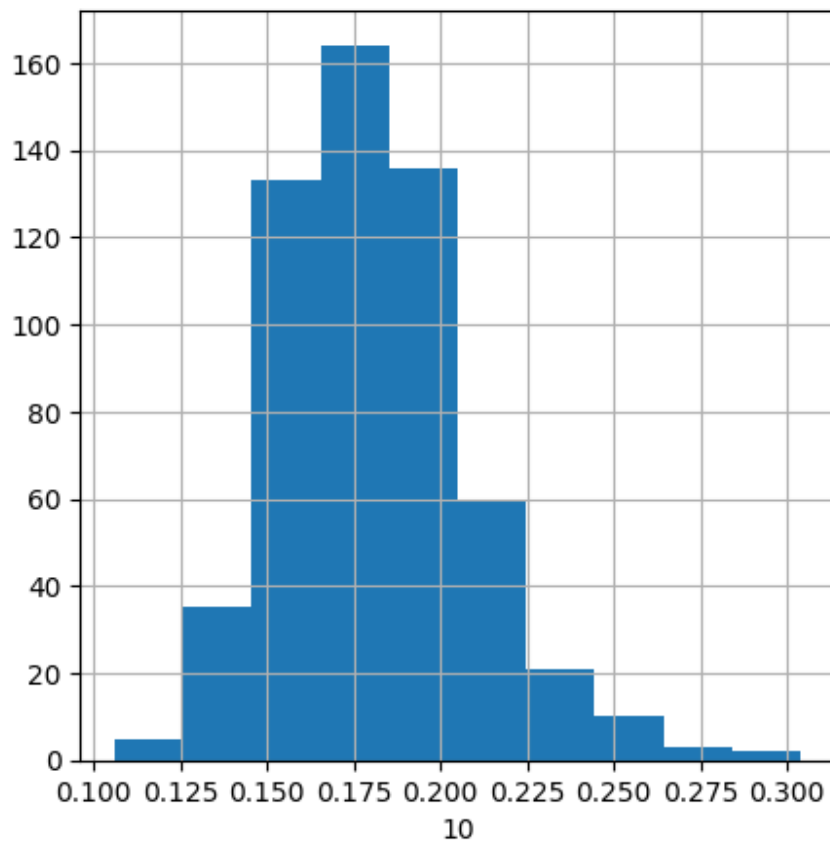
```
In [47]: for col in columns:  
         histogram(col)
```

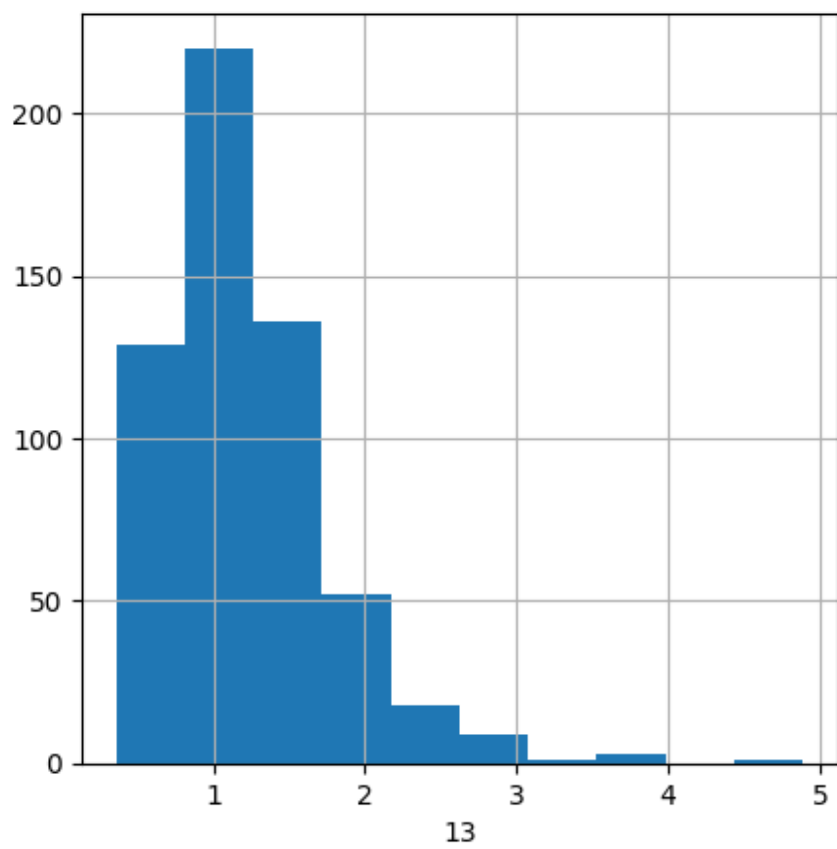
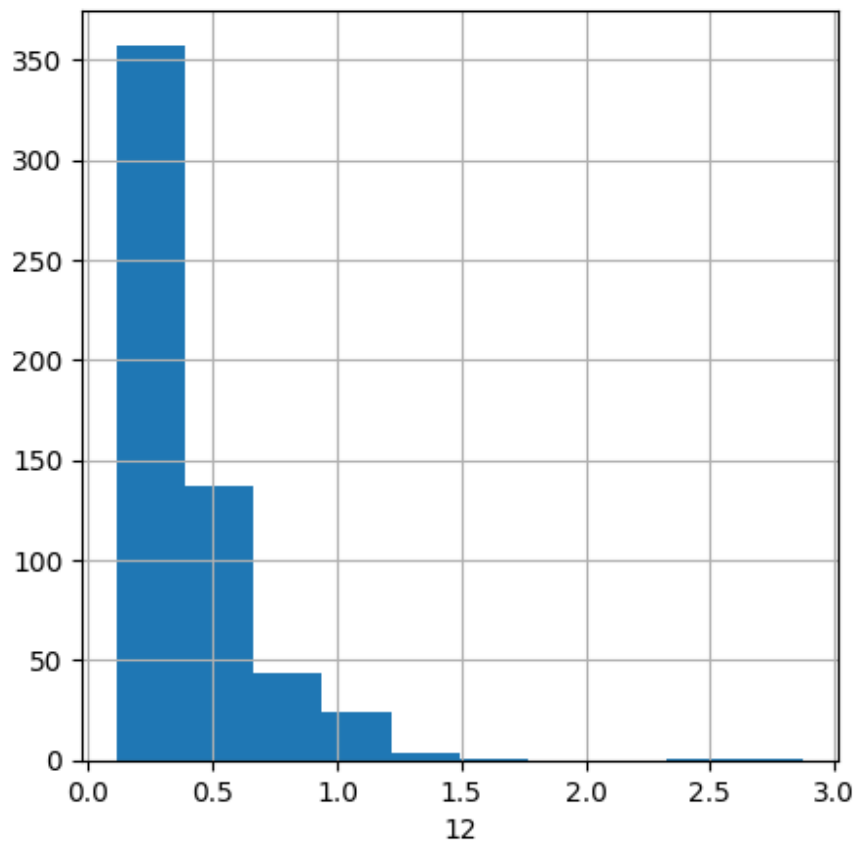


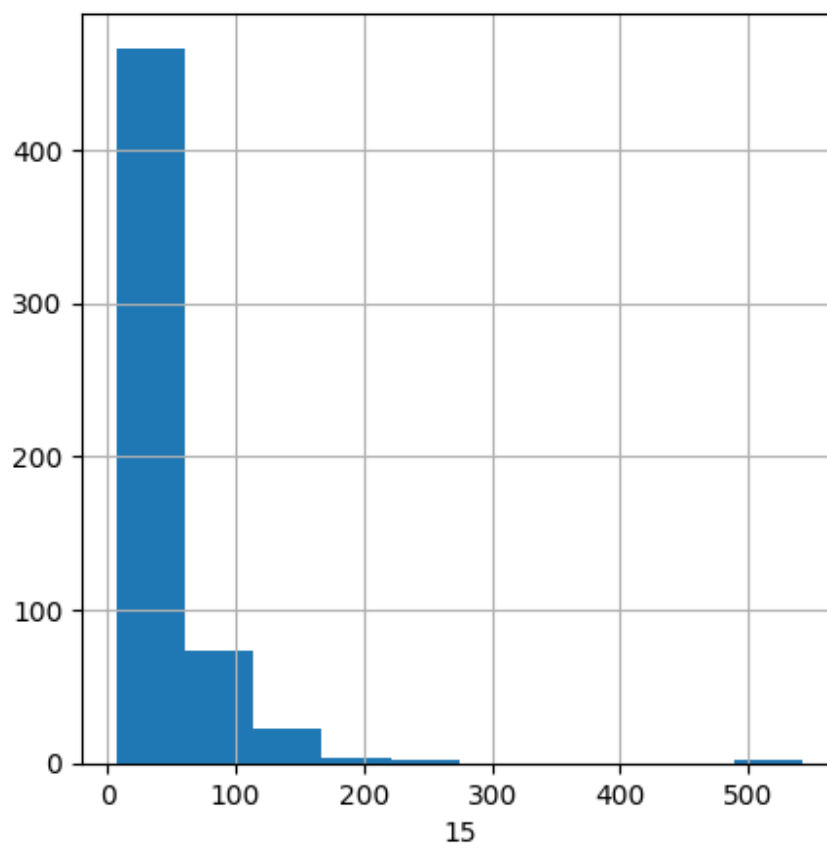
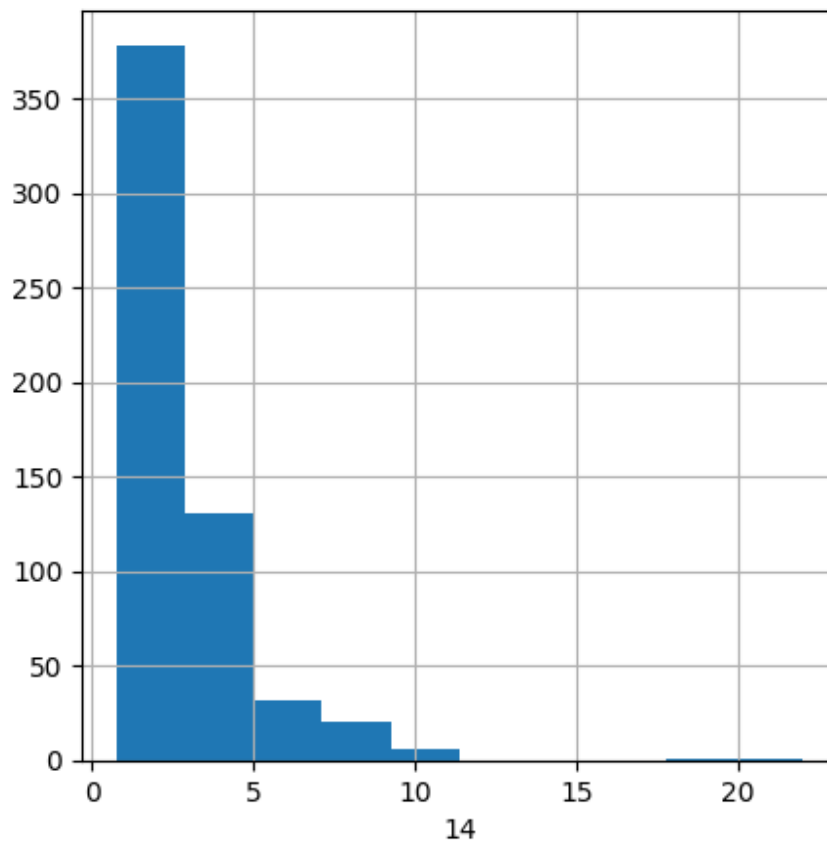


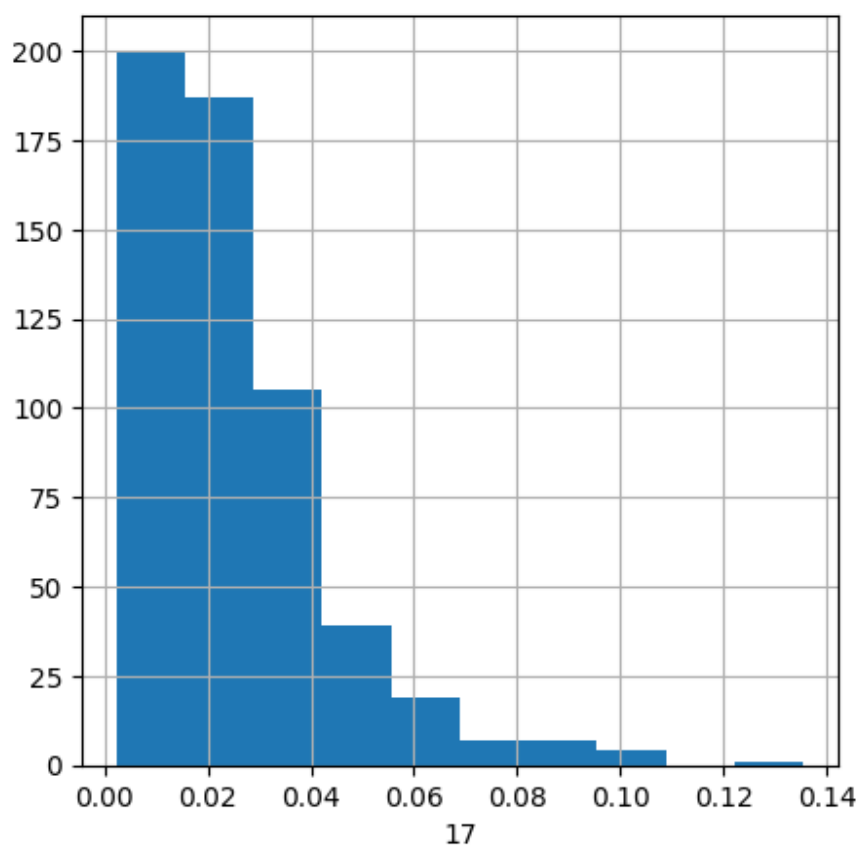
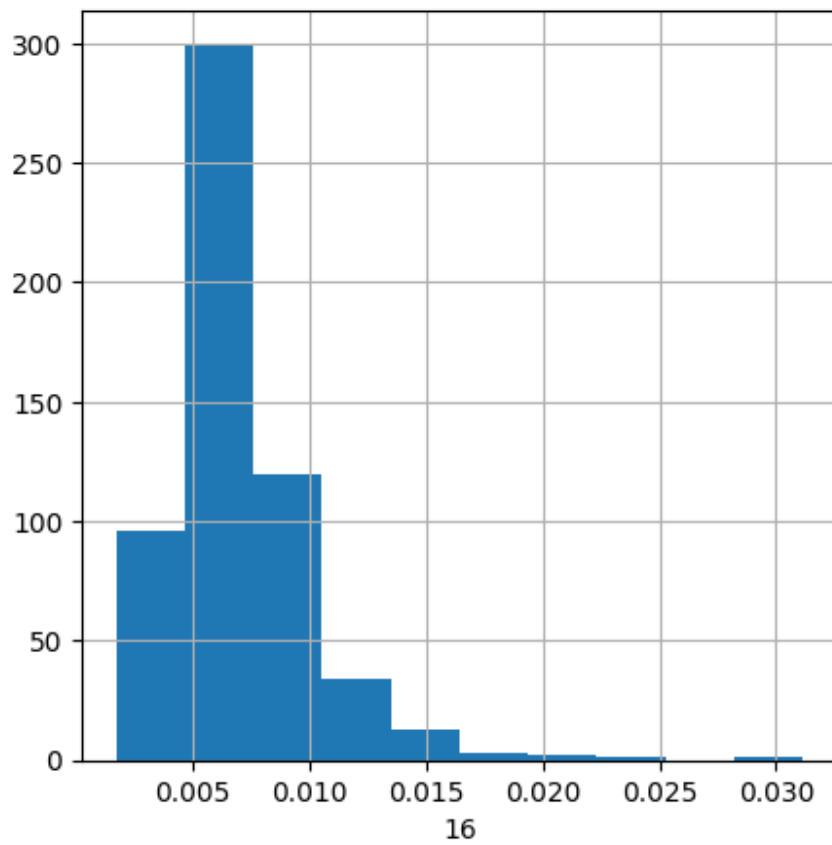


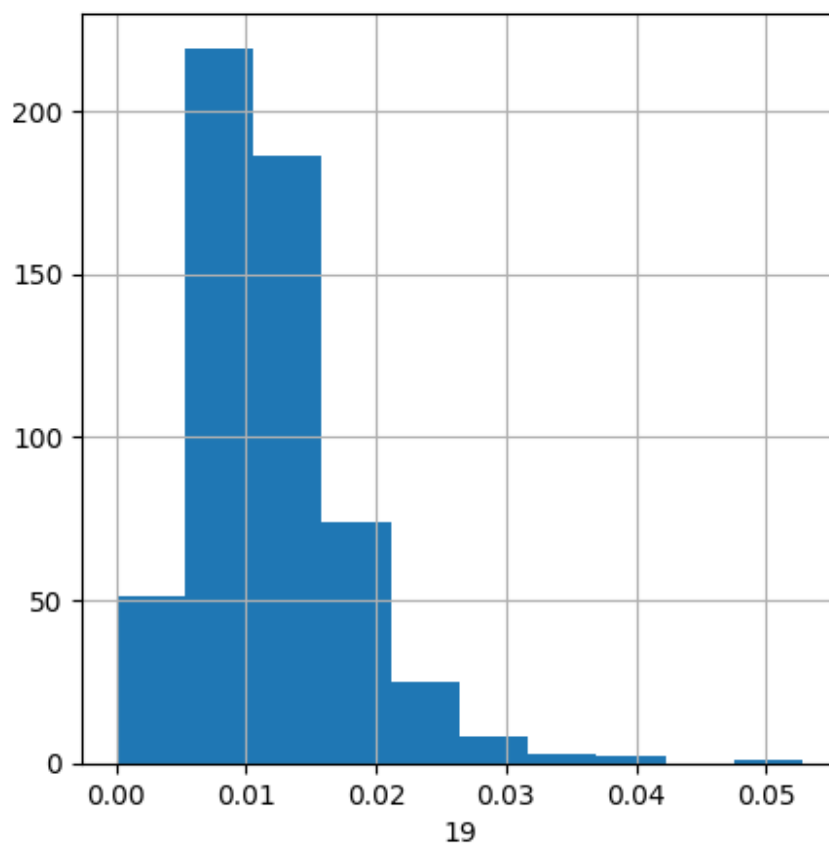
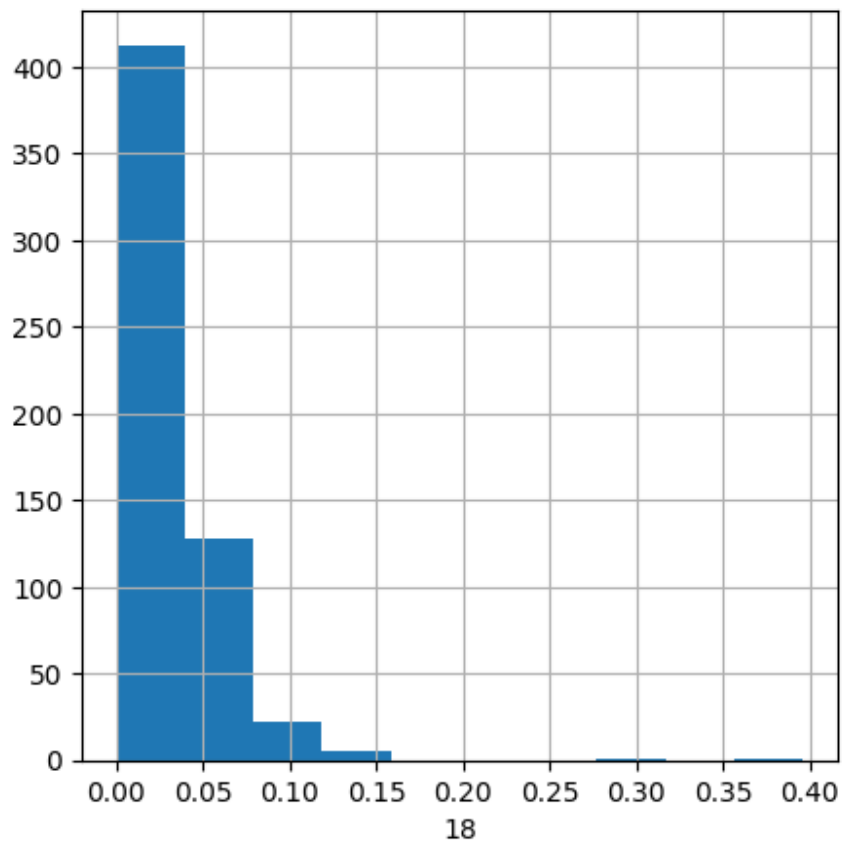


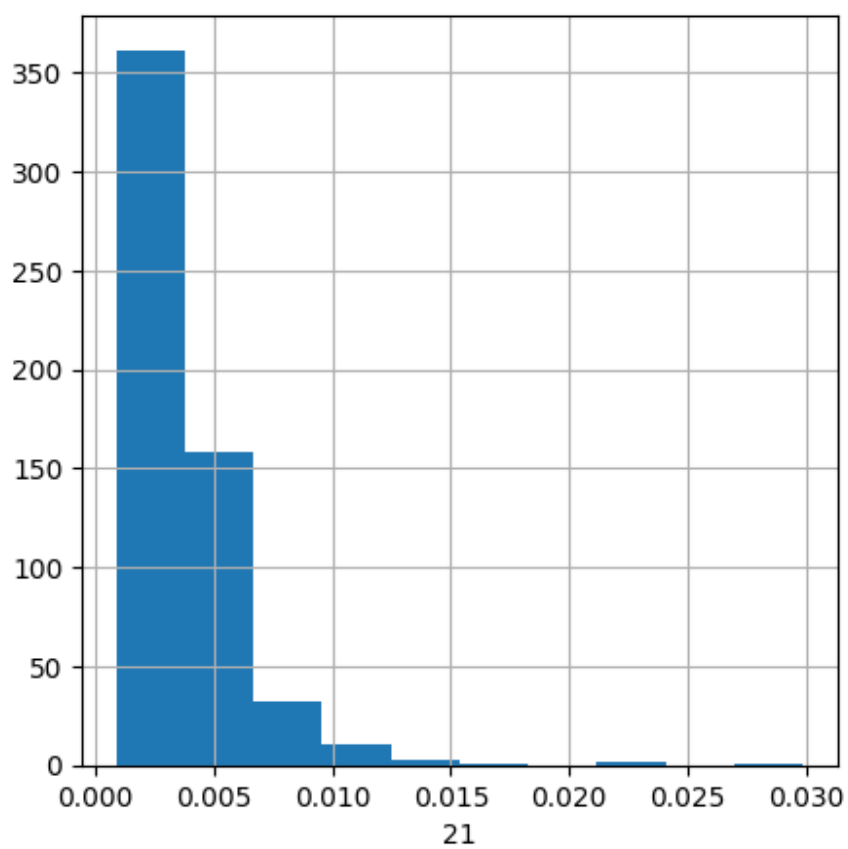
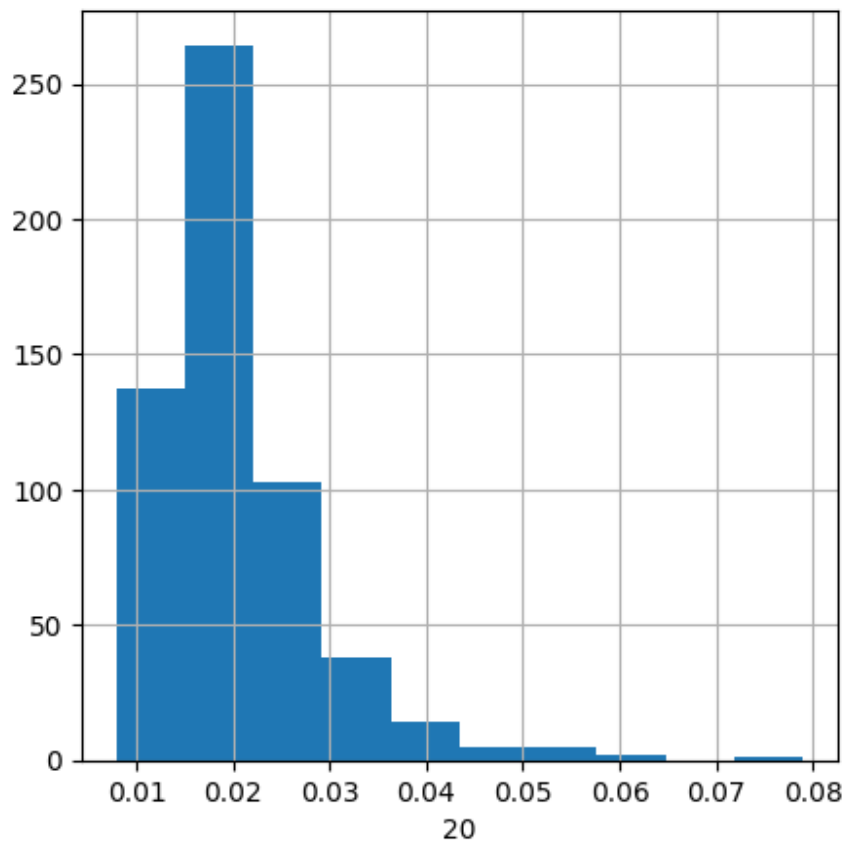


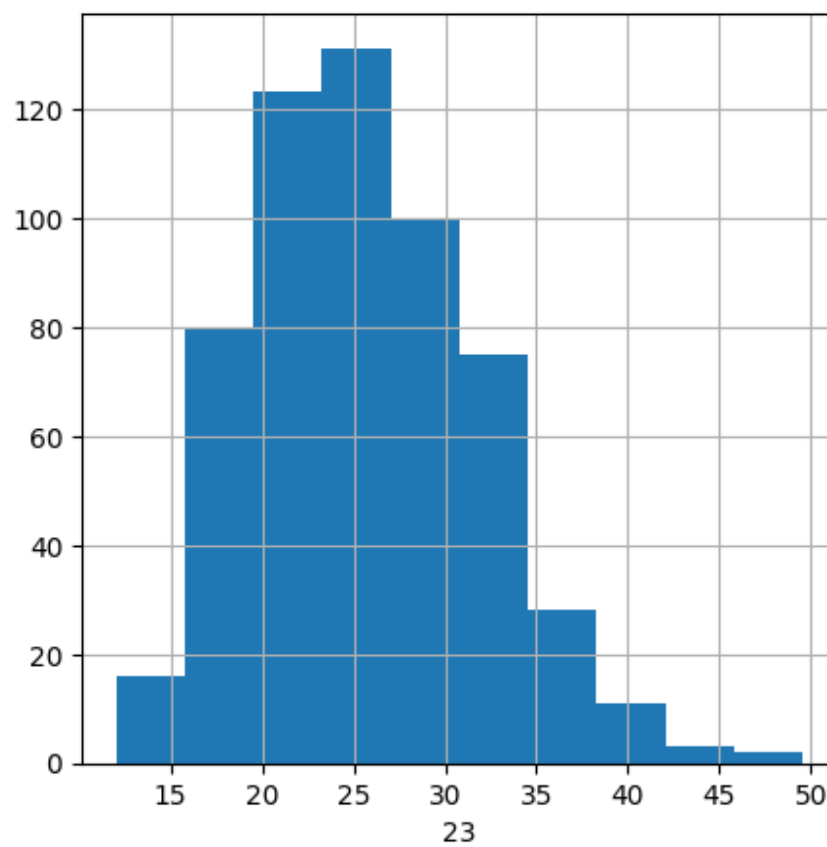
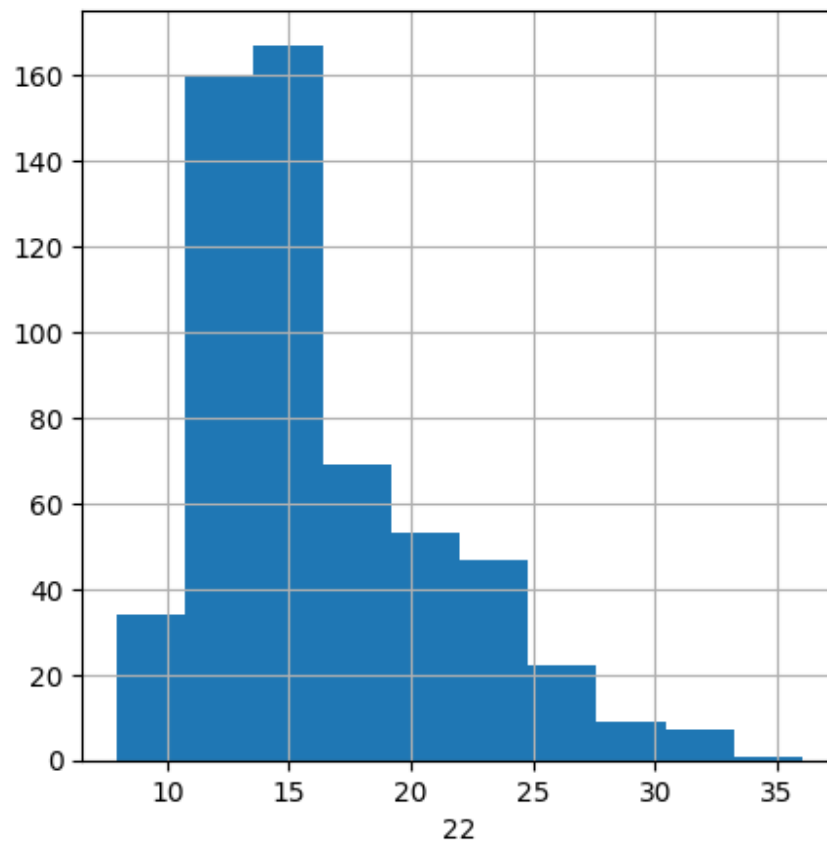


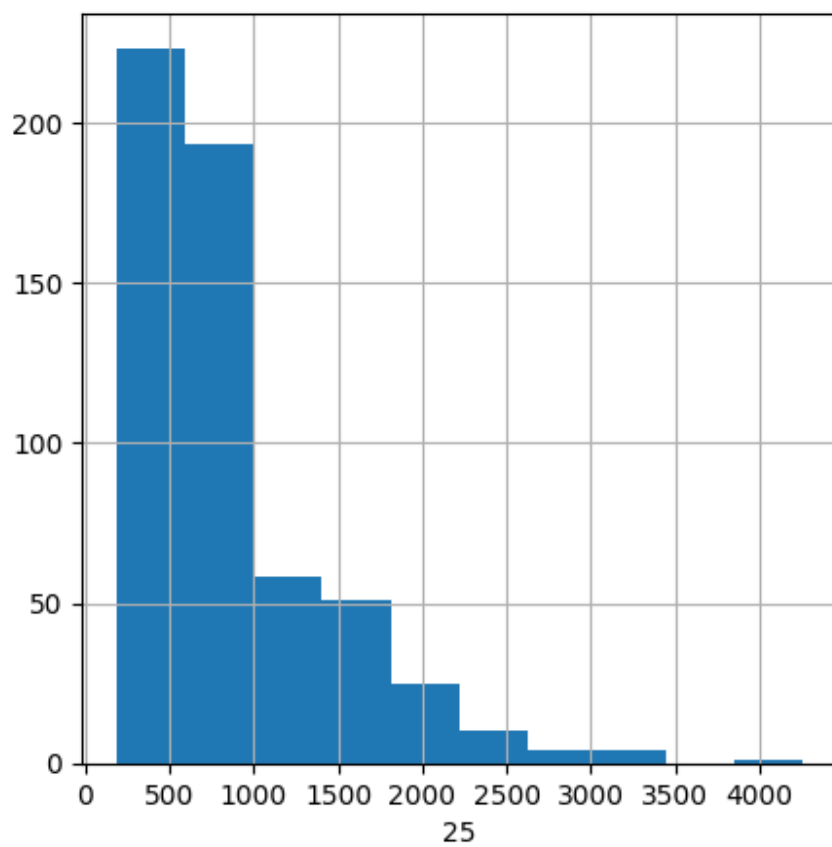
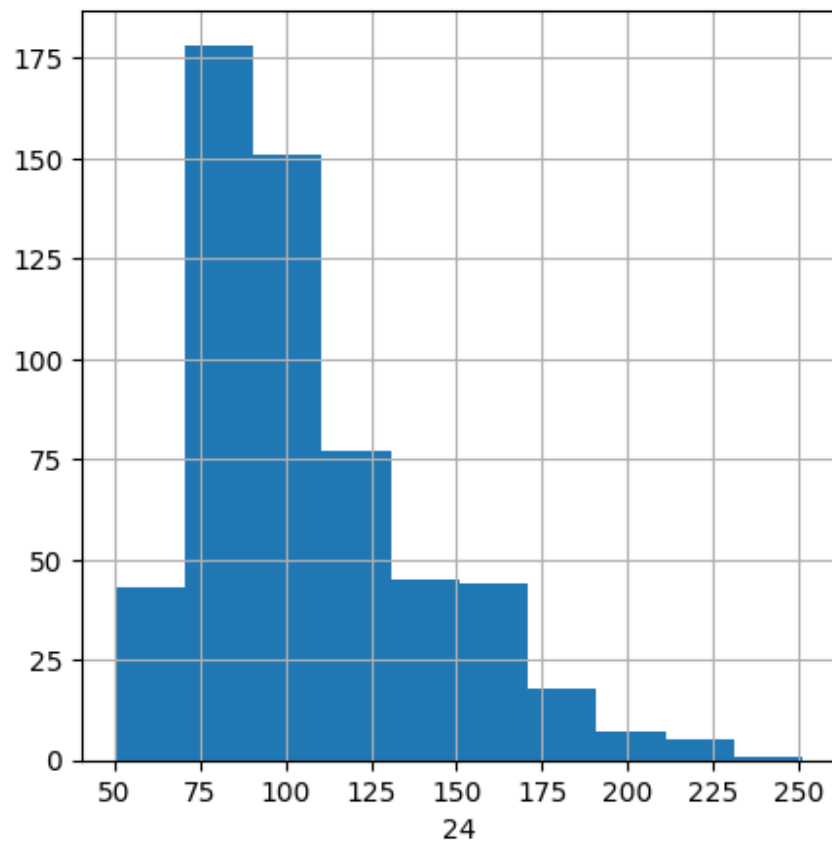


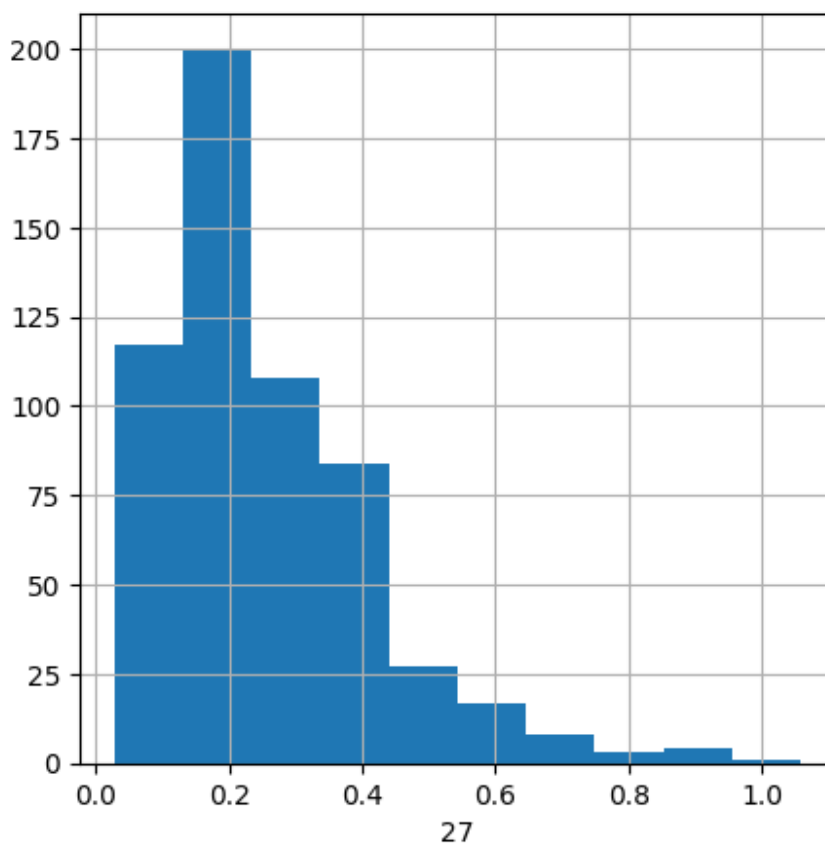
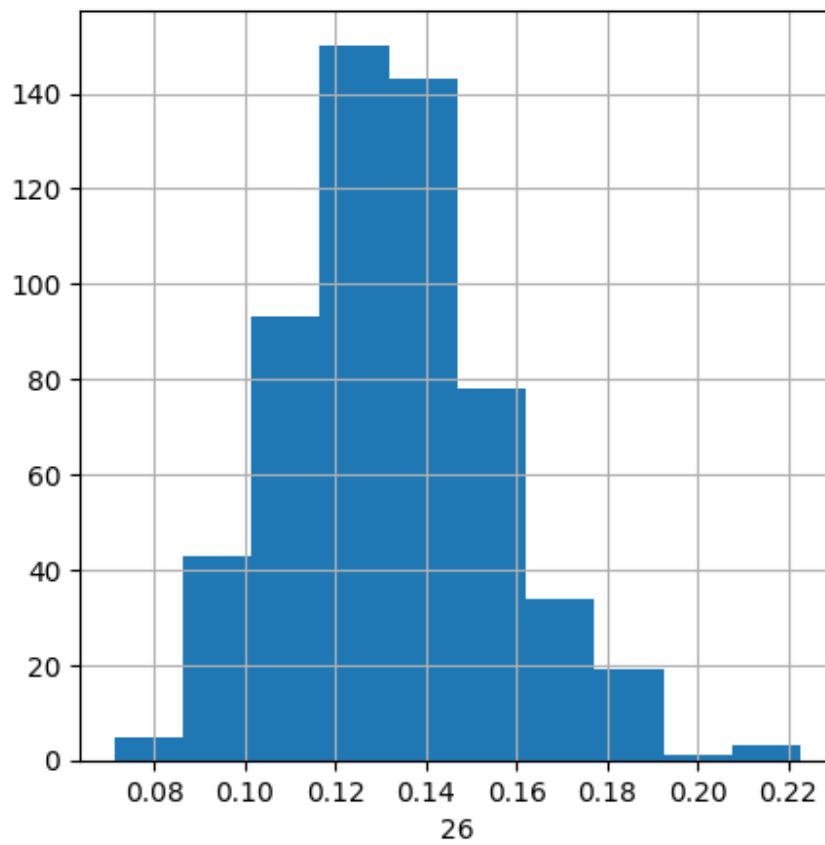


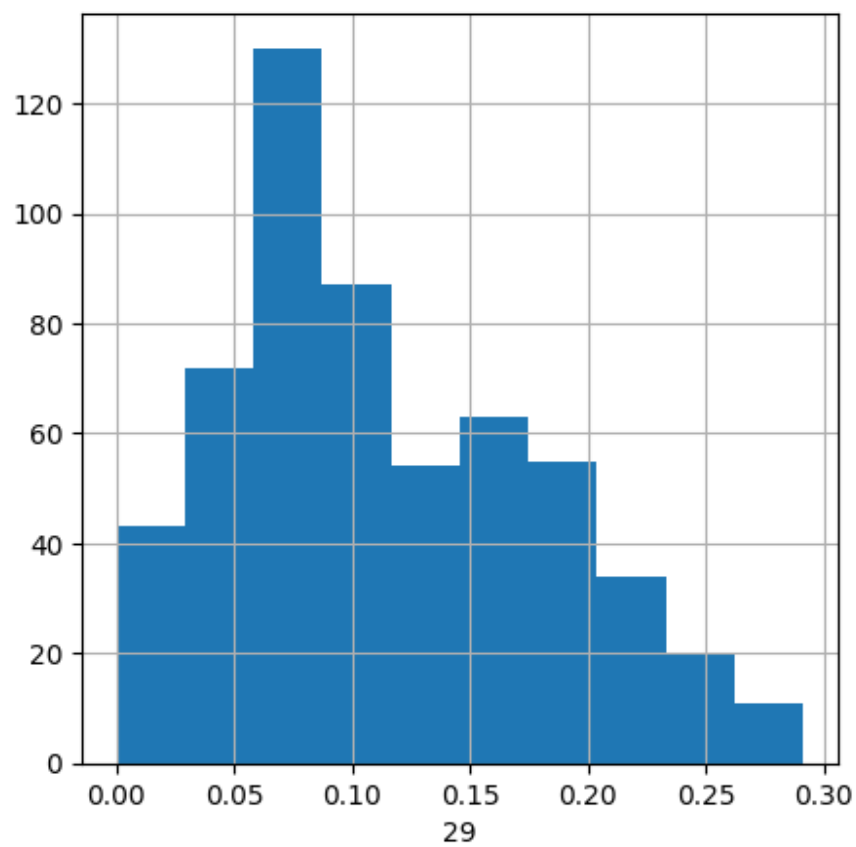
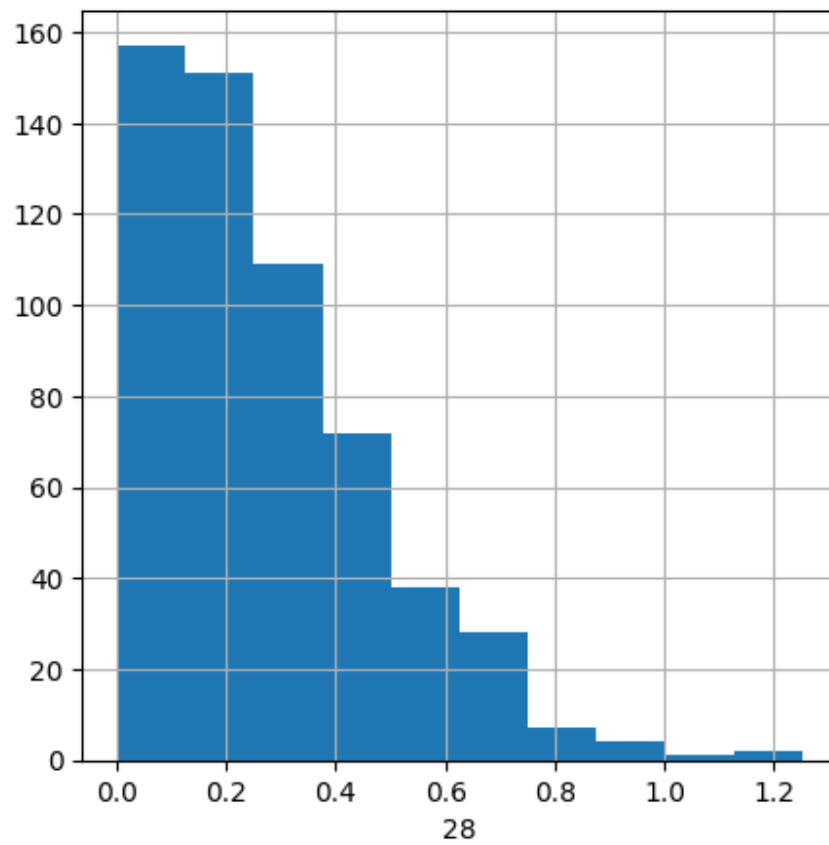


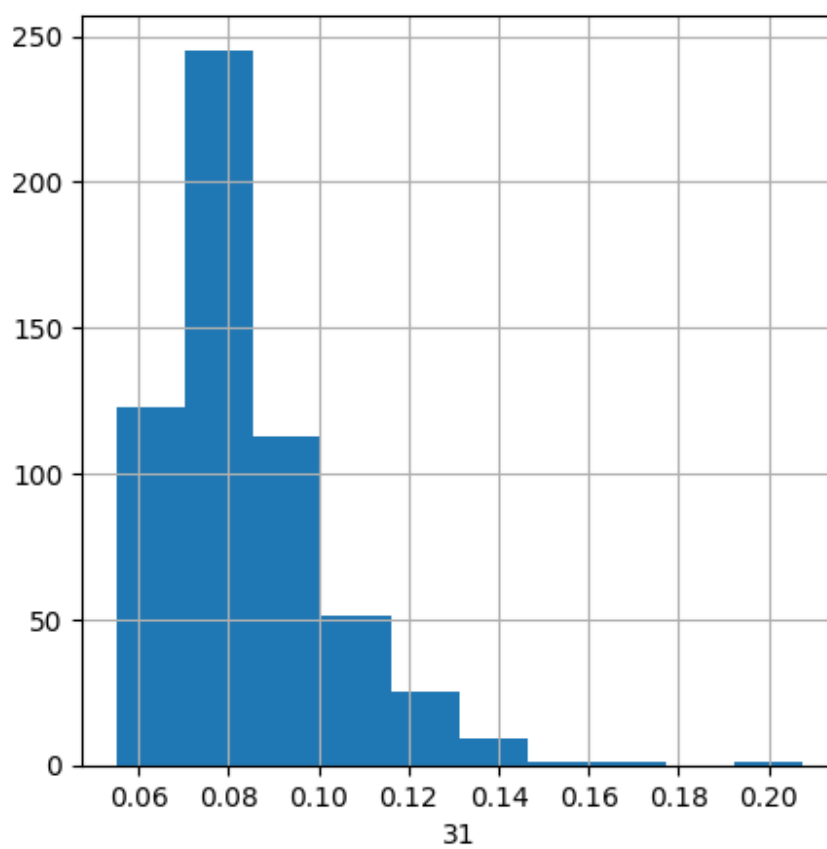
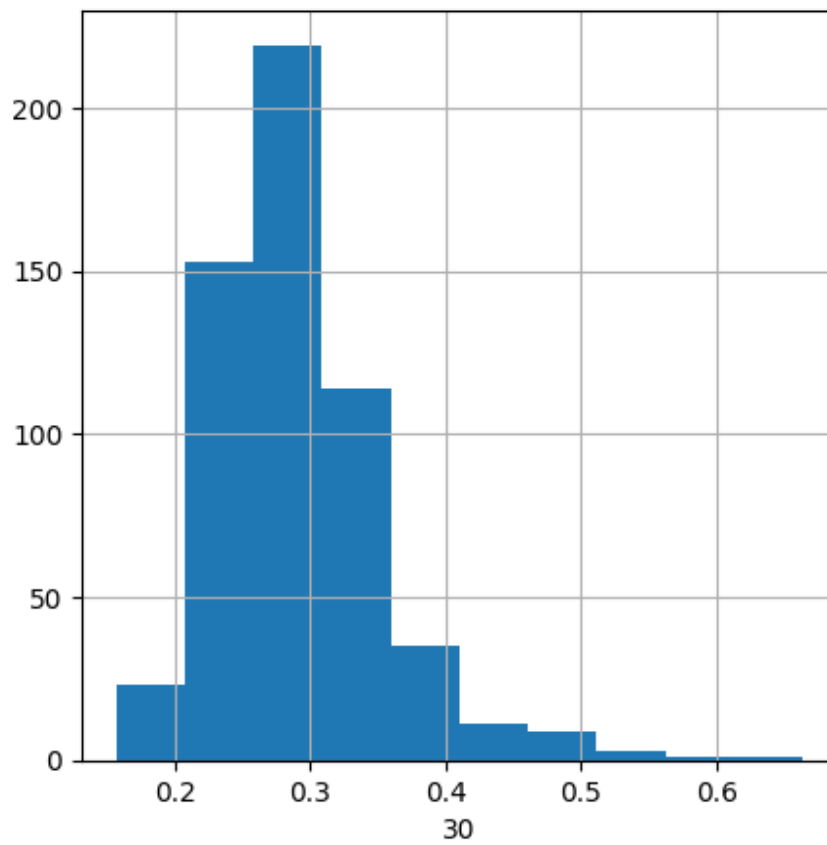




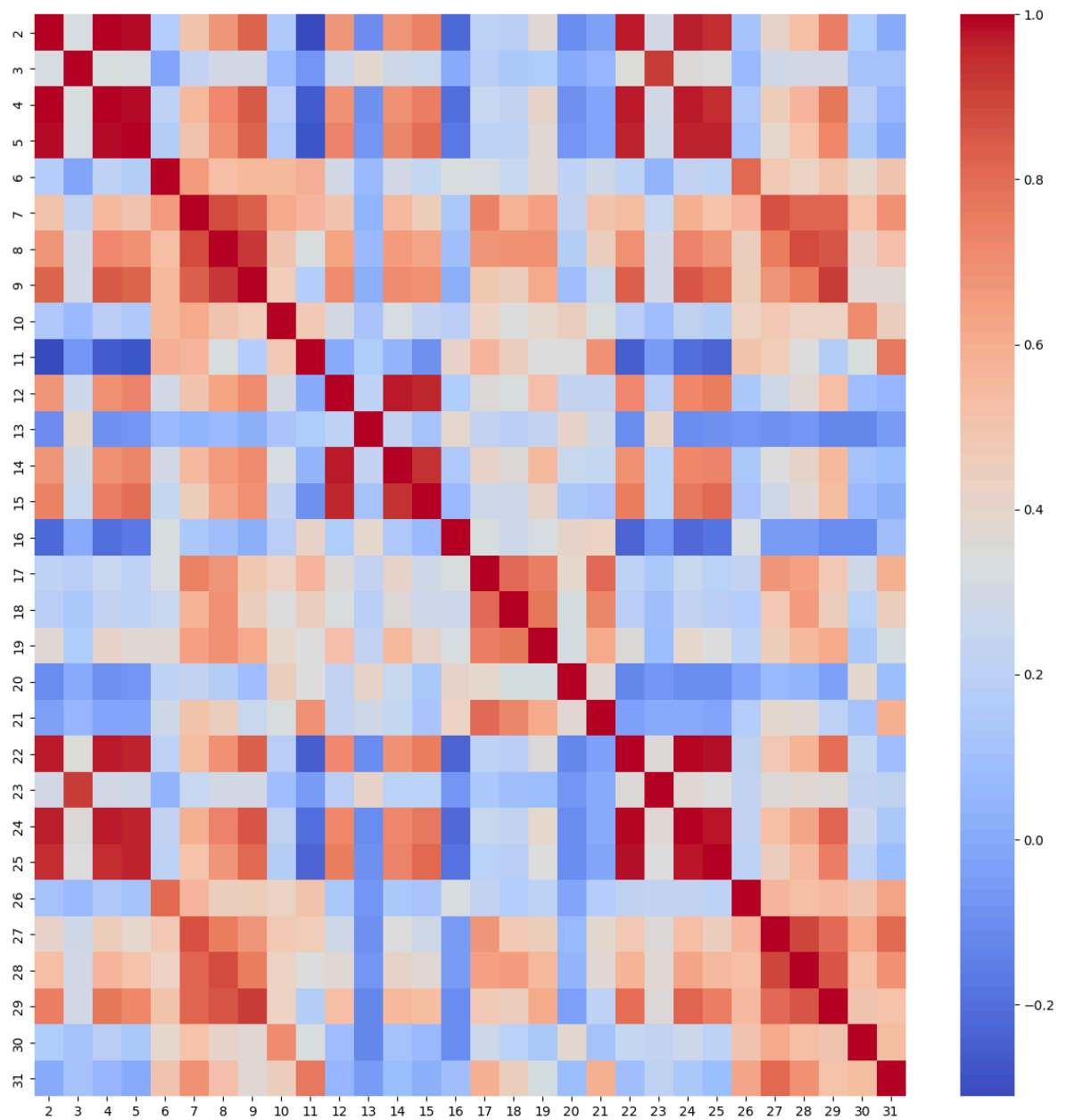








```
In [48]: correlation_analysis(columns)
```



```
In [49]: df[1]=df[1].map({
           'M':1,
           'B':0
         })
df.head()
```

Out[49]:

	0	1	2	3	4	5	6	7	8	9	...	22
0	842302	1	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.14710	...	25.38
1	842517	1	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.07017	...	24.99
2	84300903	1	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.12790	...	23.57
3	84348301	1	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.10520	...	14.91
4	84358402	1	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.10430	...	22.54

5 rows × 32 columns



```
In [50]: x=df.drop(columns=[0,1])
         y=df[1]
```

```
In [51]: x_train,x_test,y_train,y_test=train_test_split(x,y,random_state=42,test_size=0.2,shuf
```

```
In [52]: scaler=StandardScaler()

         x_train_scaled=scaler.fit_transform(x_train)
         x_test_trnsformed=scaler.transform(x_test)
```

Bagging

```
In [53]: bagging = BaggingClassifier(
         estimator=DecisionTreeClassifier(),
         random_state=42
       )

       param_grid = {
         'n_estimators': [10, 50, 100],
         'max_samples': [0.5, 0.7, 1.0],
         'max_features': [0.5, 0.7, 1.0]
       }

       grid = GridSearchCV(
         bagging,
         param_grid,
         cv=5,
         scoring='accuracy'
       )

       grid.fit(x_train,y_train)

       print(grid.best_params_)
       print(grid.best_score_)

{'max_features': 0.5, 'max_samples': 0.7, 'n_estimators': 50}
0.9692307692307693
```

Ada Boost

```
In [54]: ada = AdaBoostClassifier(  
    random_state=42  
)  
  
param_grid = {  
    'n_estimators':[50,60,70],  
    'learning_rate':[0.01,0.1,1]  
}  
  
ada_grid = GridSearchCV(  
    ada,  
    param_grid,  
    cv=5,  
    scoring='accuracy'  
)  
  
ada_grid.fit(x_train,y_train)  
  
best_ada = ada_grid.best_estimator_  
  
print("Best Parameters:", ada_grid.best_params_)  
print("Best Accuracy:", ada_grid.best_score_)
```

Best Parameters: {'learning_rate': 1, 'n_estimators': 60}
Best Accuracy: 0.964835164835165

Gradient Boosting

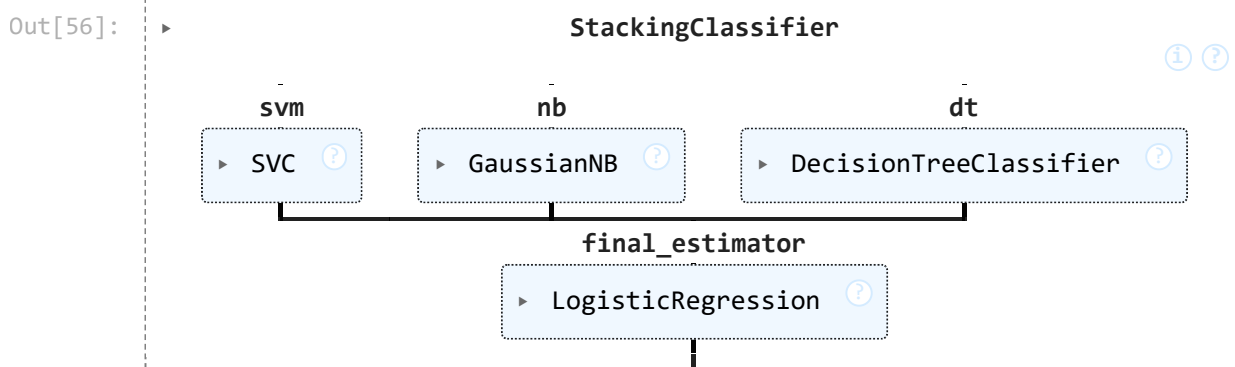
```
In [55]: gb = GradientBoostingClassifier(  
    random_state=42  
)  
  
param_grid = {  
    'n_estimators':[50,60,70],  
    'learning_rate':[0.01,0.1,1],  
    'max_depth':[1,3,5]  
}  
  
gb_grid = GridSearchCV(  
    gb,  
    param_grid,  
    cv=5,  
    scoring='accuracy'  
)  
  
gb_grid.fit(x_train,y_train)  
  
print("Best Parameters:", gb_grid.best_params_)  
print("Best Accuracy:", gb_grid.best_score_)  
  
best_gb = gb_grid.best_estimator_
```

Best Parameters: {'learning_rate': 1, 'max_depth': 5, 'n_estimators': 50}
 Best Accuracy: 0.9670329670329672

```
In [56]: base_models = [
          ('svm', SVC(probability=True)),
          ('nb', GaussianNB()),
          ('dt', DecisionTreeClassifier())
        ]

stack_model = StackingClassifier(
    estimators=base_models,
    final_estimator=LogisticRegression()
)

stack_model.fit(x_train_scaled, y_train)
```



```
In [57]: results = []

def evaluate(model, X_test, y_test, name):

    y_pred = model.predict(X_test)

    accuracy = accuracy_score(y_test, y_pred)
    precision = precision_score(y_test, y_pred)
    recall = recall_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)

    print("\n", name)
    print("Accuracy :", accuracy)
    print("Precision :", precision)
    print("Recall :", recall)
    print("F1 Score :", f1)
    print("Confusion Matrix")

    print(confusion_matrix(y_test, y_pred))

    return [name, accuracy, precision, recall, f1]
```

```
In [62]: results.append(
          evaluate(
              grid,
              x_test,
```

```
        y_test,  
        "Bagging"  
    )  
)  
results.append(  
    evaluate(  
        best_ada,  
        x_test,  
        y_test,  
        "AdaBoost"  
    )  
)  
results.append(  
    evaluate(  
        best_gb,  
        x_test,  
        y_test,  
        "Gradient Boosting"  
    )  
)  
results.append(  
    evaluate(  
        stack_model,  
        x_test_trnsformed,  
        y_test,  
        "Stacking"  
    )  
)  
)
```

Bagging

Accuracy : 0.9649122807017544

Precision : 1.0

Recall : 0.9047619047619048

F1 Score : 0.95

Confusion Matrix

[[72 0]

[4 38]]

AdaBoost

Accuracy : 0.9824561403508771

Precision : 1.0

Recall : 0.9523809523809523

F1 Score : 0.975609756097561

Confusion Matrix

[[72 0]

[2 40]]

Gradient Boosting

Accuracy : 0.9649122807017544

Precision : 0.975

Recall : 0.9285714285714286

F1 Score : 0.9512195121951219

Confusion Matrix

[[71 1]

[3 39]]

Stacking

Accuracy : 0.9649122807017544

Precision : 1.0

Recall : 0.9047619047619048

F1 Score : 0.95

Confusion Matrix

[[72 0]

[4 38]]

Comparision Table

```
In [63]: comparison_table = pd.DataFrame(
    results,
    columns=[
        'Model',
        'Accuracy',
        'Precision',
        'Recall',
        'F1 Score'
    ]
)

comparison_table
```

Out[63]:

	Model	Accuracy	Precision	Recall	F1 Score
0	Bagging	0.964912	1.000	0.904762	0.95000
1	AdaBoost	0.982456	1.000	0.952381	0.97561
2	Gradient Boosting	0.964912	0.975	0.928571	0.95122
3	Stacking	0.964912	1.000	0.904762	0.95000

ROC curve

```

In [68]: models = [
    (grid, x_test, "Bagging"),
    (best_ada, x_test, "AdaBoost"),
    (best_gb, x_test, "Gradient Boosting"),
    (stack_model, x_test_trnsformed, "Stacking")
]

plt.figure(figsize=(8,6))

for model, X_eval, name in models:

    probs = model.predict_proba(X_eval)[:,-1]

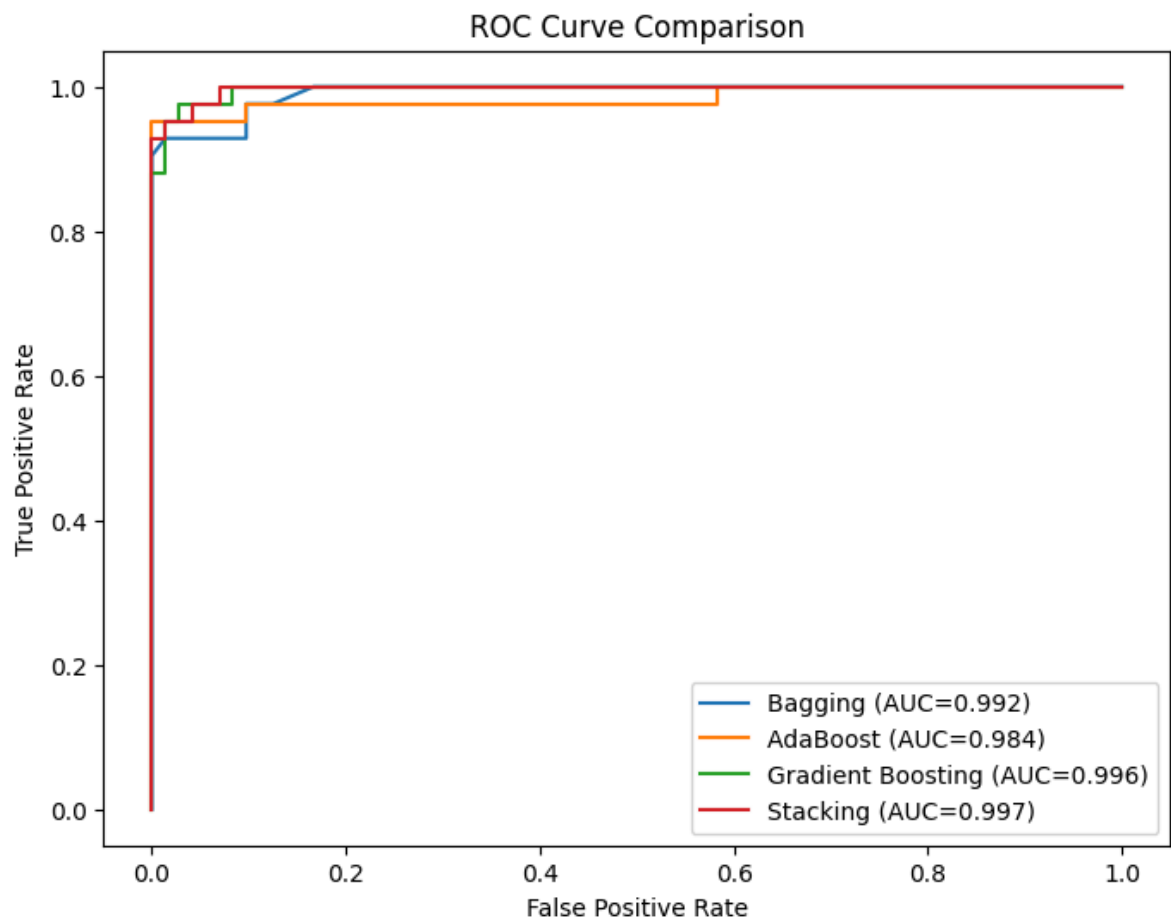
    fpr, tpr, _ = roc_curve(y_test, probs)

    auc = roc_auc_score(y_test, probs)

    plt.plot(
        fpr,
        tpr,
        label=f"{name} (AUC={auc:.3f})"
    )

plt.legend()
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve Comparison")
plt.show()

```



In []:

5 Implementation

6 Learning Outcomes

1. Learned the fundamentals of ensemble learning.
2. Understood the working of Bagging, Boosting, and Stacking.
3. Gained experience in implementing ensemble models using Scikit-Learn.
4. Learned hyperparameter tuning using GridSearchCV.
5. Evaluated classification models using multiple performance metrics.
6. Understood how ensemble methods improve prediction accuracy.